Sparse-Latent Koopman Autoencoders for Multibasin Dynamical Systems

Anonymous Authors

Abstract

Koopman autoencoders seek latent coordinates in which nonlinear dynamics evolve linearly, but multibasin systems are challenging: different basins can require incompatible local linear descriptions, so a single finite-dimensional state-reconstructing linear model may hide or average over regime structure. We show that encoders inducing latent sparsity provide latent supports as an inspectable modeling principle. We train sparse Koopman autoencoders without basin labels or trajectory-to-basin assignments, and treat the learned active latent supports as model-produced regime variables only after training. Across a suite of procedurally generated multibasin systems, merged sparse-support families recover basin structure on held-out basin interior states, achieving near-zero conditional basin uncertainty while models without sparsity incentive collapse to mixed supports. We also show that supports are functionally used rather than merely correlated with labels with intervention-based experiments. The same support family variables can select local linear predictors without oracle labels, reducing long-horizon rollout error for LISTA-style sparse encoders, while periodically decoding and re-encoding the latent state can refresh support selection after controlled basin transfer. Sparse-latent models also improve long-horizon forecasting over dense-latent controls on the multibasin systems and on chaotic flows from an external repository. These results identify sparse supports as label-free, interpretable regime variables for Koopman learning in nonlinear systems with multiple local dynamical laws.

1 Introduction

Learning representations for nonlinear dynamical systems remains a central problem in machine learning, scientific computing, and control. A particularly useful goal is to embed nonlinear dynamics into coordinates in which the evolution is linear, or at least locally linear, because linear models can be combined with mature tools for prediction, estimation, and control, including model predictive control and linear quadratic regulation (Mayne et al., 2000; Grüne and Pannek, 2017; Korda and Mezić, 2018; Kalman, 1960; Mamakoukas et al., 2019). This goal is also well motivated theoretically. Classical dynamical-systems results provide local topological linearizations near hyperbolic fixed points (Grobman, 1959; Hartman, 1960), while Koopman eigenfunction constructions can yield linearizing coordinates on basins of attraction for stable equilibria and periodic orbits (Lan and Mezić, 2013; Kvalheim and Revzen, 2021). Thus, nonlinear systems may admit linear dynamics with a specific change of coordinates.

The difficulty is that these results are primarily existential. They establish when useful coordinates can exist, but they do not by themselves provide a finite-data procedure for constructing such coordinates. Koopman operator theory formalizes the linearization idea by lifting nonlinear state evolution to a linear operator acting on observables (Koopman, 1931; Koopman and Neumann, 1932; Mezić, 2005; Brunton et al., 2022). This viewpoint has inspired data-driven approximations of Koopman dynamics, such as DMD and eDMD (Schmid, 2010; Williams et al., 2015, 2016), and

more recently to Koopman autoencoders, which learn an encoder from state space to a latent representation, a linear latent transition matrix, and a decoder back to state space (Takeishi et al., 2017; Lusch et al., 2018; Otto and Rowley, 2019; Azencot et al., 2020; Shi and Meng, 2022; Fathi et al., 2024). These models provide an appealing route from existence results to practical learning: rather than hand-designing observables, they attempt to learn the embedding directly from trajectory data.

A central challenge for Koopman approximations is that many practically interesting nonlinear systems typically contain multiple equilibria or basins of attraction; for example, a damped mechanical system can settle into different wells, a biological or chemical model can approach different stable equilibria, and a controlled system can move between regions with distinct local dynamics. However, theory shows that these systems cannot admit a single finite-dimensional global linearization with a continuous encoder-decoder pair (Lan and Mezić, 2013; Brunton et al., 2016; Pan and Duraisamy, 2024; Liu et al., 2025), and empirical evidence supports this in practice (Fathi et al., 2024). In such multistable systems, it is possible to linearize the dynamics within basins, but the linearizations needed in different basins of attraction are generally incompatible with each other. Therefore, for multi-attractor systems, the right objective is not to force all basins through one continuous global linearization, but to discover and use distinct local linearizations.

This paper proposes inducing sparse latent structure as a practical and interpretable method for Koopman learning of multi-attractor systems. By inducing the state to activate only a small subset of coordinates in lifted latent space, we can incentivize different regions of state to occupy different active coordinates, since nearby states should have similar latent embeddings, and thus enable local linearizations. In this sparse representation, nonzero coefficient values in a latent vector can provide continuous coordinates for prediction within a selected region, while the active indices provide a discrete support object that can be inspected across states and used to select local linearizations.

Sparse coding and LISTA-style encoders provide the right inductive bias for this purpose. They naturally induce piecewise-linear, union-of-subspaces structure (Olshausen and Field, 1996; Chen et al., 1998; Donoho and Elad, 2003; Gregor and LeCun, 2010), which is well-matched to this setting where different attractors or basins require different local linearizations in theory. We hypothesize that sparse-latent Koopman autoencoders will assign local dynamical regimes to distinct latent support families, yielding a learned regime variable without basin supervision. Then, periodically decoding and re-encoding the state can refresh the quality of the latent embedding over time by re-selecting the correct local dynamics when trajectories move across the state space (Fathi et al., 2024).

Contributions. This paper contributes an empirical and mechanistic study showing (1) induced latent sparsity produces support objects that are basin-aligned, (2) support objects can identify and select appropriate local region predictors without oracle labels, (3) periodic re-encoding updates (Fathi et al., 2024) can refresh the selected support after controlled basin transfer, and (4) sparse-latent Koopman autoencoders improve long-horizon forecasting performance. The results demonstrate basin identification is an *emergent* property of sparse-latent Koopman autoencoders that is useful for prediction.

2 Problem formulation

The introduction motivates a multibasin version of Koopman learning. Given sampled trajectories of a nonlinear system, the aim is to learn a state-reconstructing latent representation whose evolution is linear, without being told which basin or local dynamical regime generated each observation. We

formulate the problem at the cadence of the stored benchmark observations. For each system, let $\mathcal{X} \subseteq \mathbb{R}^{d_x}$ be the observed state space and let

$$x_{k+1} = F_{\Delta t}(x_k), \qquad x_k \in \mathcal{X}, \tag{1}$$

where $x_k \in \mathbb{R}^{d_x}$ is the state at the kth stored sample, $F_{\Delta t}$ denotes the transformation to the next discrete observation step based on underlying dynamics, and Δt is the system-specific observation interval. Forecasting is therefore defined by repeatedly applying the map $F_{\Delta t}$; a horizon of h means h applications of $F_{\Delta t}$; we denote k compositions as $F_{\Delta t}^{(k)}$. Because Δt may differ across systems, equal values of h may yield different physical time.

Basins of attraction. Many systems of interest are multistable: different initial conditions can approach different long-run behaviors. A fixed point is a state x^* satisfying $F_{\Delta t}(x^*) = x^*$. More generally, a trajectory may approach an attractor A, such as a stable equilibrium, limit cycle, countable finite set, or chaotic invariant set. The basin of attraction of A is the set of initial conditions whose trajectories converge to A:

$$\mathcal{B}(A) = \left\{ x_0 \in \mathbb{R}^{d_x} : \inf_{a \in A} \left\| F_{\Delta t}^{(k)}(x_0) - a \right\|_2 \to 0 \text{ as } k \to \infty \right\}. \tag{2}$$

When multiple attractors coexist, their basins partition the state space up to basin boundaries. This defines the setup for this paper. The task is to learn a representation that can support linear prediction where the appropriate linear law depends on where the state lies in the multibasin geometry.

Koopman theory. The stored-step map $F_{\Delta t}$ induces a Koopman operator $\mathcal{K}$ on scalar observables $g : \mathcal{X} \to \mathbb{R}$ by composition, $(\mathcal{K}g)(x) = g(F_{\Delta t}(x))$. Although $F_{\Delta t}$ may be nonlinear, $\mathcal{K}$ is linear on the space of observables. A finite-dimensional Koopman approximation searches for a vector observable $\Phi : \mathcal{X} \to \mathbb{R}^{d_z}$ and a matrix K such that $\Phi(F_{\Delta t}(x)) \approx K\Phi(x)$, while retaining enough information to reconstruct the state. Additional background is provided in section L.

Koopman autoencoders. We study this setting with Koopman autoencoders (KAEs). The model learns an encoder $E_\theta : \mathbb{R}^{d_x} \to \mathbb{R}^{d_z}$, a decoder $D_\phi : \mathbb{R}^{d_z} \to \mathbb{R}^{d_x}$, and a latent linear transition matrix $K \in \mathbb{R}^{d_z \times d_z}$. For observation x_k at arbitrary timestep k, the corresponding open-loop h-step forecast is

$$\hat{x}_{k+h} = D_\phi\Big(K^h E_\theta(x_k)\Big). \tag{3}$$

The same matrix K is applied at every step of the rollout, so any state-dependent structure required for prediction must be encoded in the learned representation rather than in a supplied regime label.

Training windows. Training uses windows of adjacent stored observations. For a batch of B windows of rollout length L, $\{x_{b,k}, x_{b,k+1}, \ldots, x_{b,k+L}\}_{b=1}^{B}$, where b indexes windows and ℓ indexes offsets within a window, the model encodes each observed state, rolls out from the initial encoded state, and decodes both reconstructed observations and latent forecasts. The encoded/reconstructed quantities are defined for $\ell = 0, \ldots, L$, while rollout/forecast quantities are defined for $\ell = 1, \ldots, L$:

$$z^{\text{enc}}_{b,k+\ell} = E_\theta(x_{b,k+\ell}), \quad z^{\text{roll}}_{b,k+\ell} = K^\ell z^{\text{enc}}_{b,k}, \quad x^{\text{rec}}_{b,k+\ell} = D_\phi(z^{\text{enc}}_{b,k+\ell}), \quad \hat{x}_{b,k+\ell} = D_\phi(z^{\text{roll}}_{b,k+\ell}). \tag{4}$$

The training losses described later are built from these reconstructed and rolled-out quantities. Each application of K is interpreted as one stored benchmark step, matching the observation map in equation (1).

Model discovers basins unsupervised. In the intended training and deployment setting, the sparse-support mechanism is not given basin labels, the number of basins, or trajectory-to-basin assignments. When benchmark basin labels are available, they are reserved for post-hoc evaluation; they are not used to choose support families, train the encoder or transition, or select supports for support-conditioned predictions. The block-diagonal and soft-block transition rows are architecture diagnostics: for those ablations only, the fixed number of latent transition groups is set from benchmark generator metadata, as detailed in section A.

3 Method

3.1 Sparse supports as local regime variables

Finite dimensional Koopman representations of multi-basin systems described in section 2 should: (i) have continuous coordinates that linearize each basin of attraction, and (ii) represent each basin of attraction with a discrete indication of which local linearization is active. Specifically, if the Koopman representation of the original state x is given by a sparse code z then the support (non-zero elements) can serve to identify the attractor, and the continuous *values* describe the coordinates of its corresponding linearized dynamics.

Given an overcomplete dictionary D, the Lasso loss (Beck and Teboulle, 2009) finds the fewest nonzero coefficients of z to reconstruct the input x. In other words, it finds the sparsest solution to the underdetermined problem.

$$z^\star(x) = \arg\min_{z \in \mathbb{R}^{d_z}} \frac{1}{2} \|x - Dz\|_2^2 + \lambda_{\text{sc}} \|z\|_1 , \qquad \lambda_{\text{sc}} > 0. \tag{5}$$

The classical solver to this problem (ISTA (Beck and Teboulle, 2009)) can be unrolled into a deep network of depth L, where each loop (layer) corresponds to a single iteration of ISTA. This is known as learned ISTA or LISTA (Gregor and LeCun, 2010), and the sparse code is given by the encoder $z = E_\theta(x)$ where the *support* is the set of active latent coordinates. With a linear decoder (unit norm columns), the support selects which decoder columns participate in reconstructing the state, while the nonzero coefficients give coordinates within that selected reconstruction subspace. If both D and z are optimized jointly (Sparse Coding (Olshausen and Field, 1996)), this can be interpreted as partitioning the input space into distinct geometric regions, where each region is mapped to a specific subset of active dictionary atoms that provide a localized linear reconstruction of the signal. Thus, by combining the Koopman and Sparse Coding objectives if different basins require different local linear descriptions, a sparse Koopman Autoencoder can represent the current state by both a continuous code and a discrete active-coordinate pattern. We therefore posit that combining Koopman representations and sparse coding implicitly captures multi-basin attractor dynamics without explicit labels, where the discrete support set identifies the active basin of attraction and the continuous coefficient values provide its localized linear representation.

We therefore evaluate the following chain:

$$\text{sparse latent code} \implies \text{support family–basin alignment} \implies \text{support-conditioned prediction.} \tag{6}$$

The sparse latent code is induced by the architecture and objective. Basin labels, when available in benchmarks, are used only after training to evaluate alignment or define diagnostic interventions.

3.2 Koopman autoencoder and sparse encoder

The predictor follows the Koopman-autoencoder formulation in section 2: an encoder E_θ, a decoder D_ϕ, and a latent transition K produce the rollout in equation (3). For the linear-decoder rows

used here, the learned parameter tuple is (θ, D, K): encoder parameters θ, a decoder dictionary D with normalized columns, and the latent transition. Thus $D_\phi(z) = Dz$, sparsity is imposed on the learned latent representation $z_t = E_\theta(x_t)$, and we vary whether K is dense, block diagonal, or softly block-regularized. Sparsity is not an assumption about the physical state x_t.

Sparse encoding. Our encoders implement the sparse-code bias through shrinkage and a training ℓ_1 sparsity penalty rather than by solving equation (5) at evaluation time. Our main sparse encoder is a learned iterative shrinkage-thresholding architecture, or LISTA encoder (Gregor and LeCun, 2010). It first computes a dense pre-code $c_t = f_\theta(x_t)$, then applies learned shrinkage refinements for $q = 0, \ldots, Q-1$:

$$u_t^{(0)} = \mathrm{shrink}(c_t, \tau), \qquad u_t^{(q+1)} = \mathrm{shrink}(Su_t^{(q)} + c_t, \tau), \qquad z_t = u_t^{(Q)}. \tag{7}$$

Here $\mathrm{shrink}(a, \tau) = \mathrm{sign}(a)\max(|a| - \tau, 0)$ is applied elementwise, S is learned, and τ controls the threshold scale for shrinkage. The thresholding step creates exact zeros, so the encoder explicitly selects active latent coordinates while learning the selection rule from the prediction objective.

Transition families. We vary the structure of K separately from the encoder. The dense transition leaves K unrestricted. The block-diagonal transition partitions latent coordinates into M fixed groups and constrains

$$K_{\mathrm{block}} = \mathrm{blkdiag}(K_1, \ldots, K_M). \tag{8}$$

We also use a soft-block LISTA ablation that keeps K dense but penalizes cross-group entries,

$$\mathcal{L}_{\mathrm{offblock}} = \sum_{i,j:\, g(i) \neq g(j)} |K_{ij}|, \tag{9}$$

where $g(i)$ is the fixed architectural group containing coordinate i.

3.3 Training objective

Training uses adjacent observation windows. For a batch of B windows with prediction horizon L, the model produces reconstructed future states $x^{\mathrm{rec}}_{b,k+\ell}$, latent rollouts $z^{\mathrm{roll}}_{b,k+\ell}$, encoded future latents $z^{\mathrm{enc}}_{b,k+\ell}$, and decoded rollout predictions $\hat{x}_{b,k+\ell}$ as defined in equation (4). The rollout is seeded by the initial state, and the following sums are over offsets $\ell = 1, \ldots, L$:

$$\bar{\mathcal{L}}_{\mathrm{pred}} = \frac{1}{BL\sqrt{d_x}} \sum_{b,\ell} \|\hat{x}_{b,k+\ell} - x_{b,k+\ell}\|_2, \qquad \bar{\mathcal{L}}_{\mathrm{rec}} = \frac{1}{BL\sqrt{d_x}} \sum_{b,\ell} \left\|x^{\mathrm{rec}}_{b,k+\ell} - x_{b,k+\ell}\right\|_2, \tag{10}$$

$$\bar{\mathcal{L}}_{\mathrm{lin}} = \frac{1}{BL} \sum_{b,\ell} \left\|z^{\mathrm{roll}}_{b,k+\ell} - z^{\mathrm{enc}}_{b,k+\ell}\right\|_2, \qquad \bar{\mathcal{L}}_{\mathrm{sp}} = \frac{1}{BL} \sum_{b,\ell} \left\|z^{\mathrm{roll}}_{b,k+\ell}\right\|_1. \tag{11}$$

The observation-space terms are normalized by $\sqrt{d_x}$ so that their scale is comparable across systems of different state dimension. The latent terms are left in their native scale because the latent dimension is fixed within each comparison.

The total objective is

$$\mathcal{L} = \frac{1}{L}\left(\lambda_{\mathrm{pred}}\bar{\mathcal{L}}_{\mathrm{pred}} + \lambda_{\mathrm{rec}}\bar{\mathcal{L}}_{\mathrm{rec}} + \lambda_{\mathrm{lin}}\bar{\mathcal{L}}_{\mathrm{lin}} + \lambda_{\mathrm{sp}}\bar{\mathcal{L}}_{\mathrm{sp}}\right) + \mathcal{L}_{\mathrm{struct}}. \tag{12}$$

Here $\mathcal{L}_{\mathrm{struct}} = 0$ for the dense and block-diagonal transition variants. For the soft-block transition, $\mathcal{L}_{\mathrm{struct}} = \lambda_{\mathrm{off}}\mathcal{L}_{\mathrm{offblock}}$, which probes whether a soft transition grouping is sufficient to induce useful

support structure. The prediction and latent-linearity terms make the representation useful for Koopman rollout, the reconstruction term keeps the code tied to the observed state, and the sparsity term encourages selective activation.

3.4 Support objects

Given $z = E_\theta(x) \in \mathbb{R}^{d_z}$, a *support rule* first converts the latent code into a Boolean active-coordinate vector $m(x) \in \{0,1\}^{d_z}$, where $m_i(x) = 1$ means that latent coordinate i is active for state x. Equivalently, the exact support is the index set $S(x) = \{i : m_i(x) = 1\}$. Exact supports defined by a threshold can be too fine-grained: weak coordinates may cross the threshold, and a single basin may be covered by several nearby or partially overlapping active-coordinate vectors.

We therefore distinguish exact supports from *support families*, which group nearby Boolean support vectors and test whether these groups form basin-scale objects. We use three support constructions:

- **Absolute-threshold support.** The absolute support vector is the Boolean mask

$$m_{\text{abs}}(x) = \left(\mathbf{1}\{|z_i| > 10^{-3}\}\right)_{i=1}^{d_z} \in \{0,1\}^{d_z}, \qquad S_{\text{abs}}(x) = \{i : m_{\text{abs},i}(x) = 1\}.$$

  Here $m_{\text{abs},i}(x)$ is the i-th entry of the mask, and $S_{\text{abs}}(x)$ is the equivalent active-index set. This strict active-coordinate mask is the input to F_{abs} and to canonical wrong-support interventions.

- **Absolute-threshold support family.** F_{abs} groups exact supports whose active-index sets substantially overlap. The overlap score is the Jaccard similarity, or intersection-over-union, between the active coordinates of two Boolean support vectors (Jaccard, 1901). On an evaluation collection $\mathcal{X}$, the algorithm first counts all distinct Boolean vectors and then visits those distinct vectors from most to least frequent, following a leader-style threshold clustering rule (Hartigan, 1975). Each family f stores one fixed representative vector r_f: the exact support vector that created that family. For the next distinct vector m, the algorithm computes

$$J(m, r_f) = \frac{\sum_i \mathbf{1}\{m_i = 1 \text{ and } r_{f,i} = 1\}}{\sum_i \mathbf{1}\{m_i = 1 \text{ or } r_{f,i} = 1\}},$$

  with $J(0,0) = 1$ when both vectors are all zero. If the best existing representative has Jaccard similarity at least 0.5, every occurrence of m receives that family label. Otherwise m starts a new family and becomes the representative r_f for that family. The representatives are not learned centroids and are not updated after assignment; they are exact support vectors chosen by each deterministic frequency-ordered pass. Thus the procedure is closer to threshold-based leader clustering than to k-means: the number of families is induced by the threshold and the data order, not fixed in advance. We write $F_{\text{abs}}(x)$ for the family assigned to $m_{\text{abs}}(x)$, and $|F_{\text{abs}}|$ for the number of occupied families.

- **Fixed-size top-8 support and family.** $S_{\text{top8}}(x) = \text{top}_8(|z|)$ is the set of the eight largest-magnitude coordinates. Unlike S_{abs}, it gives every state the same number of active indices and does not depend on a magnitude threshold. $F_{\text{top8}}(x)$ is obtained by applying the same Jaccard merge to exact S_{top8} support vectors. Each state still begins with exactly eight active indices; F_{top8} is a family label for nearby eight-index support vectors, not a new fixed-size support.

The main basin-alignment object is F_{abs}, which preserves the thresholded active set. The main routing object for support-conditioned prediction is F_{top8}, which gives every state the same number of active indices before family merging and is invariant to global rescalings of z when there are no top-8 ties. This is a design choice, not a reported head-to-head claim that F_{top8} is empirically

superior to F_{abs} as a router. A scale-relative rule, $S_{\text{rel}}(x) = \{i : |z_i| > 0.1 \max_j |z_j|\}$, is used only in appendix sensitivity checks for centered local laws.

3.5 Support-conditioned rollouts

Support-conditioned rollouts test whether learned support families can select useful local predictors without supplied regime labels. They are post-hoc predictors fit after the Koopman autoencoder is trained, so they evaluate the learned representation rather than introducing basin supervision into training. The comparison point is the same model's global latent rollout,

$$\hat{z}_{t+1}^{\text{global}} = Kz_t, \qquad \hat{x}_{t+1}^{\text{global}} = D_\phi(\hat{z}_{t+1}^{\text{global}}), \tag{13}$$

which uses the single learned transition K at every step. A support-conditioned rollout instead computes a route label from the current latent state and uses that label to choose the next-step latent update. We write $F_{\text{top8}}(z)$ for the same top-8 mask and family assignment applied directly to a latent vector. No basin label, basin count, or oracle transition signal is used to define the route.

The main local prediction support-conditioning object in table 2a is F_{top8}. On a disjoint fitting set, let $\mathcal{I}_c = \{t : F_{\text{top8}}(z_t) = c\}$ and let $\bar{z}_c = |\mathcal{I}_c|^{-1} \sum_{t \in \mathcal{I}_c} z_t$ be the mean current latent state for family c. For each observed family, we fit a centered ridge regression for the slope K_c:

$$K_c = \operatorname*{arg\,min}_{\widetilde{K} \in \mathbb{R}^{d_z \times d_z}} \sum_{t \in \mathcal{I}_c} \left\| (z_{t+1} - \bar{z}_c) - \widetilde{K}(z_t - \bar{z}_c) \right\|_2^2 + \eta \left\| \widetilde{K} \right\|_F^2. \tag{14}$$

Equivalently, equation (14) minimizes the one-step error of the affine map

$$T_c(z) = \bar{z}_c + K_c(z - \bar{z}_c) = K_c z + (I - K_c)\bar{z}_c$$

on transitions whose current latent state has route c. Thus the centering does not introduce a second model: it fixes the intercept of the route-local affine predictor to $(I - K_c)\bar{z}_c$ after fitting the centered slope. During rollout, $c_t = F_{\text{top8}}(z_t)$ is recomputed from the current predicted latent state, and the next latent is initialized to the global prediction $\hat{z}_{t+1}^{\text{global}}$. If the route was sufficiently represented in the fitting set, the next latent is overwritten by applying the fitted local map,

$$\hat{z}_{t+1}^{\text{local}} = T_{c_t}(z_t) = \bar{z}_{c_t} + K_{c_t}(z_t - \bar{z}_{c_t}). \tag{15}$$

Exact S_{top8}-local routing uses the same equations with $c_t = S_{\text{top8}}(z_t)$ and is an appendix ablation.

3.6 Periodic re-encoding

If a support indicates the currently relevant local dynamics, a long rollout should be able to update that support. We therefore evaluate periodic re-encoding, following Fathi et al. (2024). Starting from $z_k = E_\theta(x_k)$ and a re-encoding period m, the model advances for m Koopman steps, decodes the predicted state, and encodes that prediction again:

$$\tilde{z}_{k+m} = K^m z_k, \qquad \tilde{x}_{k+m} = D_\phi(\tilde{z}_{k+m}), \qquad z_{k+m} = E_\theta(\tilde{x}_{k+m}). \tag{16}$$

The same procedure is repeated over the rollout horizon. Each rollout uses only the model's predicted state; it does not use the true future state, a basin label, or an oracle for transitions between basins.

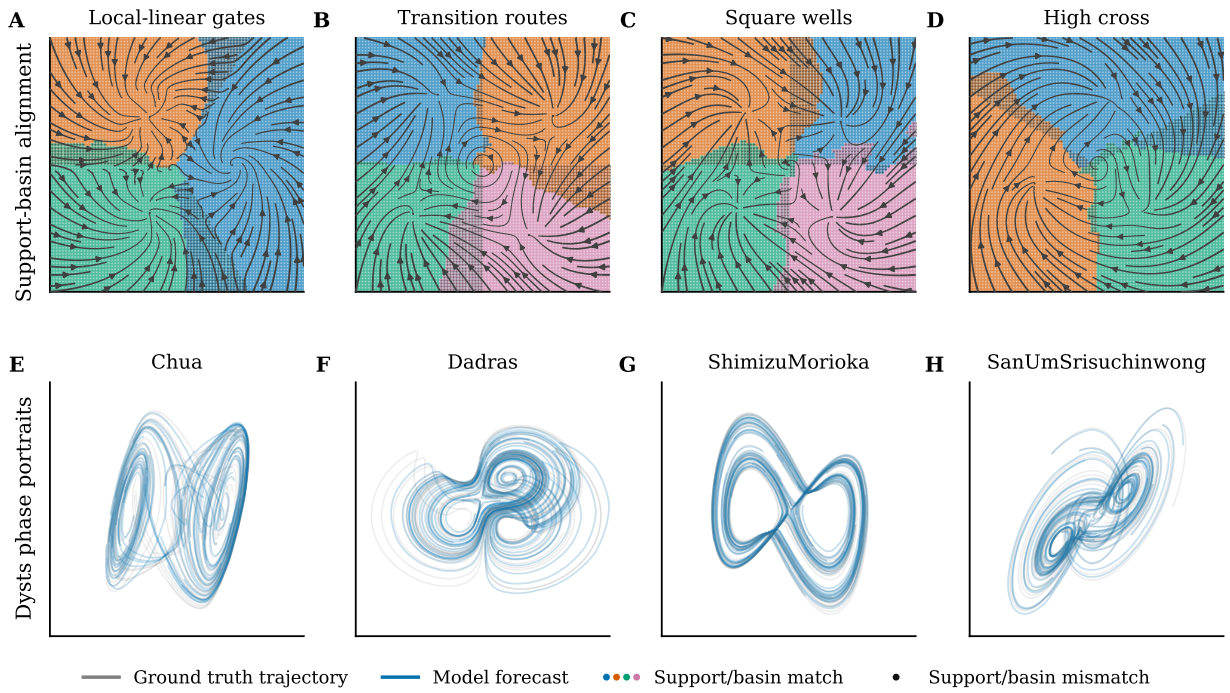


Figure 1: Benchmark visual summary. Top: procedurally generated multibasin systems where gray streamlines are the true vector field, colored dots mark support-family/basin matches, and black dots mark support-family/basin mismatches. The learned LISTA F_{top8} support families are mapped post hoc to each family's dominant evaluation basin. Bottom: $dt\times30$ Dysts horizon 5000 phase portraits with held-out truth in gray and forecasts from the best seed-0 LISTA variant.

4 Experimental procedure

The experiments follow the chain equation (6). We test alignment of support families F_{abs} with withheld basin labels. We then test whether supports are functionally used in prediction and whether support families can route local forecasts without oracle labels, whether periodic re-encoding refreshes S_{top8} support after basin transfer, and finally examine long-horizon forecasting of sparse-latent models in procedurally generated multibasin systems and chaotic flows without basin labels.

Benchmarks and training. The main benchmark for our interpretability experiments contains 15 procedurally generated two-dimensional multibasin systems, including multiwell potentials, gated local-linear systems, depth cascades, and polygonal and transition-rich layouts. Basin labels are available only for evaluating alignment and diagnostic metrics. We also use a subset of 10 chaotic systems from the Dysts repository (Gilpin, 2021, 2023) with the step size dt multiplied by a factor of 30 as an additional forecasting test. The retained-system inventory and analytic generator families are summarized in section C. We compare the six model rows shown in table 1: three LISTA sparse encoders (two of which have structured latent transitions), two sparse-latent MLP controls applying the sparsity penalty $\bar{\mathcal{L}}_{\text{sp}}$ (11) with dense or block-diagonal transitions, and a dense-latent MLP control with no sparsity incentive. Each model is trained for 15 random seeds on each system. Training uses broad online rollout windows from the benchmark generator, with an additional hard-reset pool to oversample boundary and transient states; it is not restricted to the deep-state slice used for the support-alignment diagnostic. Exact training details are reported in section A.

Forecasting and statistics. Forecasting is evaluated on all held-out rollout initial states and their trajectories at the stored observation cadence, not only on deep states. It is summarized by mean squared error (MSE) at prespecified horizons. Periodic re-encoding results report the best value over a fixed refresh grid, where the rollout uses only predicted states and never the true future trajectory.

Point estimates are interquartile means (IQM) over seeds within each system, then arithmetic means across systems. We use seed IQMs within each system rather than the arithmetic mean because few-run deep-learning evaluations are sensitive to outlier runs, and IQM is a robust aggregate that reduces uncertainty in benchmark comparisons (Agarwal et al., 2021). Horizon-trend lines use the same point estimate as the tables, with seed-bootstrap bands that keep the benchmark systems fixed and summarize uncertainty over seeds for an average system.

We also perform rigorous statistical significance tests. For multibasin forecasting and support diagnostics, per-system significance counts use one-sided paired Wilcoxon signed-rank tests on matched seeds with Holm correction across systems. For support-based local predictor routing and Dysts forecasting, seeds are first summarized within each system and the confirmatory tests use benchmark systems as the independent units. Additional details on statistical testing and system-by-system performance are reported in sections A, F, H and K.

Support alignment. The support-alignment experiment tests a static mechanistic question: after label-free training over the multibasin state space, do sparse support families behave like basin-interior regime variables? If a support family indexes the local chart or basin-specific description being used by the model, then on states safely inside a basin it should leave little uncertainty about the withheld benchmark basin label. States near separatrices are a different object: there, support changes, shared supports, or extra transition supports can be mechanistically appropriate rather than failures of basin-interior alignment. We therefore evaluate this diagnostic on held-out deep states, not because the model was trained only there, but because basin interiors are the cleanest population for testing whether supports identify basin-scale regimes.

For a state x, let $d_1(x)$ and $d_2(x)$ be its distances to the nearest and second-nearest benchmark attractor centers; the basin-depth margin is $d_2(x) - d_1(x)$. The main table uses the *global deep slice*: the top quartile of this margin over all held-out evaluation states in a system. This slice is evaluation-only and uses benchmark geometry only to choose clean basin-interior states for scoring. A per-basin top-quartile slice, where each basin contributes its own relative-deep states, is a basin-stratified appendix robustness check rather than the main Table 1 support-diagnostic population. The main metrics are $H(B \mid F_{\mathrm{abs}})$, which should be small when support families are basin-aligned, and $\overline{|F_{\mathrm{abs}}|}$, a non-directional family-count diagnostic compared with the number of basins represented.

Functional use is tested by a wrong-support ablation. For each testable system, we construct a canonical deep-slice S_{abs} mask per basin. Systems whose deep slice contains only one basin are excluded. Then, a held-out rollout replaces the inferred mask with a canonical mask from another basin and holds it fixed. The reported value is the wrong-support/base error ratio.

Support-conditioned routing for prediction. The routing experiment freezes the trained representation and fits the family-local predictors in equation (14) on a disjoint set. Table 2a reports the F_{top8}-local rollout in equation (15) against the same model's global transition rollout in equation (13). The primary metric is the routed/global MSE ratio for the same trained model; values below one mean that the learned support family selects a better local predictor without oracle basin labels. Exact S_{top8} gated and local routes are retained as appendix ablations in section J, and

Table 1: Sparse-latent rows improve forecasting on the 15-system multibasin benchmark, while sparse supports align with basin interiors and wrong-support interventions show that active coordinates are used by the predictor. The forecasting columns H100 and H1000 are computed on all held-out rollouts; the support columns $H(B \mid F_{\text{abs}})$, wrong-support ratios, and $\overline{|F_{\text{abs}}|}$ are computed on the global deep-state slice. Values are arithmetic means across systems after first summarizing seeds within each system by IQM; $\overline{|F_{\text{abs}}|}$ is an arithmetic mean over per-system seed means. Brackets give within-system Wilcoxon/Holm counts. The LISTA + local K_c row freezes the LISTA encoder/decoder and reports a second-stage support-family-local transition-map forecasting variant; its support diagnostics are therefore inherited from LISTA rather than recomputed as a new representation. Arrows denote the better direction; **bold** marks the best entry for directional metrics. $\overline{|F_{\text{abs}}|}$ should be compared with the number of basin labels present in the evaluated deep slice (mean 3.00).

	Forecasting (all held-out)		Support diagnostics (global deep slice)			
Model	H100 ↓	H1000 ↓	$H(B \mid F_{\text{abs}})$ ↓	wr. h=1 ↑	wr. h=20 ↑	$\overline{\lvert F_{\text{abs}}\rvert}$
LISTA	0.0407 [15/15]	0.166 [15/15]	**0.0391** [11/11]	2.19×10^4 [11/11]	110 [10/11]	3.2
LISTA + local K_c	0.0692 [14/15]	0.139 [14/15]	same frozen encoder/support diagnostics as LISTA			
LISTA-BD	0.0411 [15/15]	0.135 [15/15]	0.0456 [11/11]	2.10×10^4 [11/11]	99.0 [10/11]	3.0
LISTA-SB	**0.0387** [15/15]	0.130 [15/15]	0.0442 [11/11]	2.60×10^4 [11/11]	156 [10/11]	3.1
Sparse MLP, BD	0.0397 [15/15]	0.117 [15/15]	0.0881 [11/12]	5.57×10^4 [11/11]	358 [11/11]	2.6
Sparse MLP	0.0397 [15/15]	**0.107** [15/15]	0.0715 [11/11]	7.08×10^4 [11/11]	483 [11/11]	2.6
Dense MLP	0.830 *[baseline]*	2.93 *[baseline]*	0.765 *[baseline]*	20.0 *[baseline]*	2.37 *[baseline]*	1.0 *[baseline]*

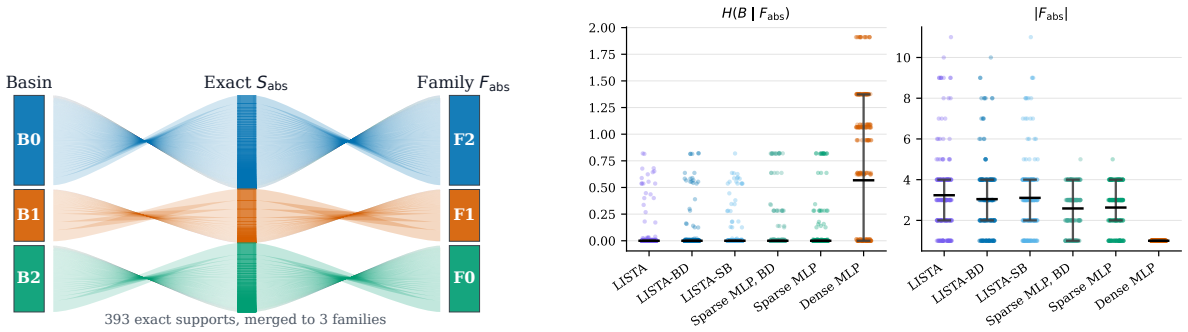
appendix controls replace learned supports with oracle basin partitions, latent clusters, or random matched partitions in section G.

Periodic re-encoding for support refresh. The periodic re-encoding support refresh experiment evaluates exact S_{top8} selection after controlled transfer into a new basin. It compares a rollout that keeps the source-basin S_{top8} mask with one that periodically decodes, re-encodes, and re-selects S_{top8}. Basin labels are used only to construct and score these transfers. The reported quantities are fraction of post-entry steps routed through the target class and the refreshed/stale-source support forecast-MSE ratio. Additional procedural details are collected in sections A and F.

5 Results

The results support four main conclusions. Sparse-latent encoders expose basin-scale support families F_{abs}; the active coordinates (S_{abs} masks) affect prediction; support families F_{top8} can select useful local predictors for LISTA-style encoders; and sparse-latent models can improve long-horizon forecasting over dense-latent models.

F_{abs} identify basin membership (table 1). LISTA F_{abs} support families have low basin uncertainty on the deep-state slice and sparse-latent MLP models also have far less basin-uncertainty than the dense-latent MLP baseline. LISTA rows expose about three support families on average, matching the retained benchmark's represented deep-slice basin-count mean/median 3.00/3, whereas the dense-latent MLP collapses the representation to one support family and mixes basins. Sparse-latent encoders therefore expose compact F_{abs} families that identify basin membership; figure 2a shows this compression for a representative system. The per-(system, seed) distributions in figure 2b show that the gap is not driven by outliers.



(a) Basin–support family alluvial.

(b) Per-seed entropy and support-family counts.

Figure 2: Support families align with basins and do so consistently across systems and seeds. **Left:** alluvial view for a LISTA KAE on local-linear gates; held-out deep states are grouped by post-hoc evaluation basin, exact S_{abs} mask, and merged F_{abs} family, with 393 exact supports merging to three families, which is one-to-one with represented basins. **Right:** per-seed deep-slice support results on the retained benchmark. Dots show system–seed pairs, gray I-bars mark first-to-third quartile ranges, and black bars mark IQM summaries for $H(B \mid F_{abs})$ and arithmetic means for $|F_{abs}|$.

Table 2: Support-conditioned prediction diagnostics. **Left:** label-free F_{top8}-local routed/global MSE ratios and system-win counts on all states; lower ratios and higher win counts are better. Superscripts denote significance tests: $*$, $p_{Holm} < 0.05$; $**$, $p_{Holm} < 0.01$. **Right:** exact S_{top8} refresh after controlled basin transfer. Target-route is the post-entry target-basin route fraction (higher is better); the MSE ratio compares refreshed with stale source supports (lower is better); brackets give Wilcoxon/Holm counts. **Bold** marks the best directional entry.

(a) F_{top8}-local routing.

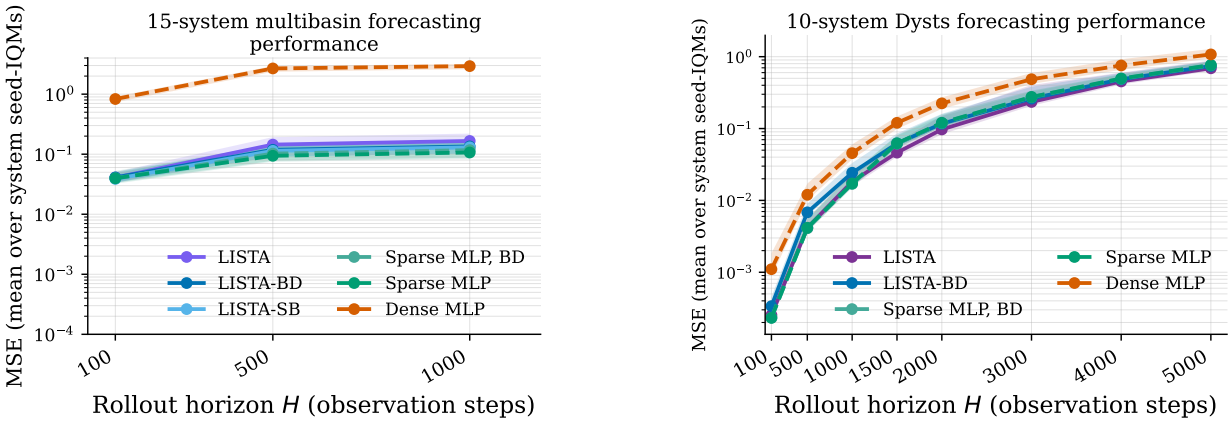
Model	$H100$		$H1000$	
	F_{top8} ratio	system wins	F_{top8} ratio	system wins
LISTA	0.239^{*}	**13/15**	$\mathbf{1.41 \times 10^{-11}}^{**}$	**15/15**
LISTA-SB	0.021^{*}	**13/15**	$2.42 \times 10^{-7*}$	14/15
LISTA-BD	$\mathbf{0.011}^{*}$	**13/15**	$6.96 \times 10^{-5*}$	**15/15**
Sparse MLP, BD	0.032^{**}	12/15	6.53×10^{-8}	**15/15**
Sparse MLP	0.038^{**}	**13/15**	5.19×10^{-4}	**15/15**
Dense MLP	154	1/15	85.7	1/15

(b) Periodic re-encoding S_{top8} support refresh.

Model	Period 1		Period 10	
	Target-route	MSE ratio	Target-route	MSE ratio
LISTA	**0.846**	0.086 [12/12]	**0.847**	0.029 [12/12]
LISTA-SB	0.812	0.125 [12/12]	0.822	0.036 [12/12]
LISTA-BD	0.84	0.154 [12/12]	0.83	0.041 [12/12]
Sparse MLP, BD	0.785	0.076 [12/12]	0.828	0.024 [12/12]
Sparse MLP	0.81	**0.073** [12/12]	0.823	**0.023** [12/12]
Dense MLP	0.743	0.229 [12/12]	0.844	0.052 [12/12]

Accurate forecasts require the correct active coordinates for a basin (table 1). The wrong-support ratios are orders of magnitude larger for sparse-latent models than for the dense-latent baseline at horizon 1, and the effect persists at horizon 20. We conclude that active coordinates are functionally coupled to prediction.

F_{top8} selects local predictors without basin labels (table 2a). F_{top8}-local routing, defined by equations (14) and (15) and compared with the global rollout in equation (13), gives strong system-level gains for LISTA-style encoders and the relative improvement increases with the forecasting horizon. Sparse MLP rows also route well, whereas dense MLPs fail under family-local routing. In section G, additional results show random matched partitions do not explain the LISTA gains, while latent clusters can be competitive for MLP controls. Exact S_{top8} and gated S_{top8} routing results are also in section J. A representative active-index codebook for F_{top8} families is shown in figure 6.



(a) 15-system multibasin benchmark.

(b) 10-system Dysts $dt\times30$ benchmark.

Figure 3: Forecasting horizon trends. Lines show arithmetic means across per-system seed-IQM MSEs on a log scale. Bands show fixed-system seed-bootstrap 95% intervals after system-wise log-relative normalization, anchored to the displayed mean, so they summarize seed uncertainty within an average system.

Table 3: Forecasting performance on the 10 chaotic flows from Dysts. Values are arithmetic means across systems of per-system seed-IQM MSEs; lower is better, and **bold** marks the best value at each horizon. Superscripts mark the system-level significance tests against the dense-latent MLP baseline, Holm-corrected across all Dysts model–horizon comparisons: $*$ indicates $p_{\text{Holm}} < 0.05$.

Model	H100	H500	H1000	H1500	H2000	H3000	H4000	H5000
LISTA	$2.53\times10^{-4*}$	0.00415*	0.0183*	**0.0462***	**0.0972***	**0.234***	**0.452***	**0.689***
LISTA-BD	$3.38\times10^{-4*}$	0.00682	0.0243*	0.0619*	0.117*	0.261*	0.485*	0.741*
LISTA-SB	4.85×10^{-4}	0.00881	0.0389	0.0973	0.165	0.393	0.686	0.997
Sparse MLP	$\mathbf{2.32\times10^{-4*}}$	0.00416*	0.0172*	0.0622*	0.120*	0.275*	0.497*	0.765
Sparse MLP-BD	$2.33\times10^{-4*}$	**0.00410***	**0.0170***	0.0615*	0.119*	0.274*	0.493*	0.760
Dense MLP	0.00110	0.0120	0.0455	0.120	0.224	0.485	0.754	1.08

Refreshing S_{top8} with periodic re-encoding tracks basin transfer (table 2b). After controlled basin transfer, S_{top8}-refreshed rollouts route most post-entry steps to the target-basin class under the evaluation-only support-to-basin mapping. Target-route fractions are 0.743–0.847, and refreshed/stale support MSE ratios are below 0.229 for every displayed row and period. Refreshing therefore helps select the appropriate current local description.

Sparse-latent models improve long-horizon forecasting of multibasin systems and chaotic flows. On the 15 controlled multibasin systems and the 10 Dysts systems, every sparse-latent model has lower aggregate MSE than the dense-latent MLP across the reported horizons (table 1, table 3 and figure 3b, figure 3a). Average-case forecasting on Dysts chaotic systems using IQMs over systems in the Appendix also shows that block-diagonal transition structure is most useful at longer horizons. We conclude that sparse-latent encoding improves long-horizon forecasting.

6 Conclusion

Multibasin dynamics make global finite-dimensional Koopman learning difficult because the useful linear description can depend on which basin or local chart is active. This paper shows that sparse-latent Koopman autoencoders can expose that dependence as an interpretable support variable learned without basin supervision. Across controlled multibasin systems, support families align with held-out basin labels, wrong-support interventions show that supports are functionally tied to prediction, support-family routing improves local rollouts, and periodic re-encoding refreshes supports after basin transfer. Sparse-latent models also retain forecasting advantages on an external chaotic-flow stress test.

The central takeaway is that sparsity is not only a regularizer for Koopman autoencoders. It can turn a continuous latent representation into a hybrid object: coefficients carry within-regime state information, while supports provide discrete, model-produced candidates for local dynamical regimes. This suggests a practical path for interpretable multi-regime Koopman learning when regime structure is not specified in advance. Future work should scale the support-family mechanism to higher-dimensional systems, learn explicit support-transition graphs, and test the same support-routed predictors in controlled systems where the local regime variable can directly influence planning.

A Additional experimental details

Training and model rows. The controlled multibasin comparison uses the six model rows reported in table 1. All use latent dimension $d_z = 256$ and are trained on rollout length $L = 8$ windows (nine adjacent stored states) for 200,000 optimization steps with minibatches of 256 windows, using the prediction, reconstruction, latent-linearity, and sparsity objective described in section 3.3. The multibasin runs use AdamW, with learning rate 5×10^{-5} for the encoder and decoder, learning rate 5×10^{-6} for the latent transition, and weight decay 10^{-4} on the non-transition parameters. In the notation of equation (12), the relative weights are $\lambda_{\text{pred}} = 1$, $\lambda_{\text{rec}} = 0.03$, and $\lambda_{\text{lin}} = 1$; sparse rows use $\lambda_{\text{sp}} = 3\times10^{-3}$, while the Dense MLP row sets $\lambda_{\text{sp}} = 0$. The block-diagonal and soft-block rows are architecture diagnostics: they use benchmark generator metadata only to set the fixed number of latent transition groups. Basin labels and trajectory-to-basin assignments are not used by the training objective, support extraction, or routing rules. All six controlled rows use the same boundary-emphasized training-state distribution. Before model training, we build a pool of difficult reset states by probing 4096 candidate initial conditions for 32 steps. Candidates are ranked by short-horizon perturbation sensitivity and residual late-rollout motion, and the top 1024 are retained. During training, 50% of rollout windows start from this retained pool with small jitter. This helps to ensure states from basin boundaries and transient regions are sufficiently covered in the training set, as otherwise they are rarely seen. This training distribution is deliberately broader than the deep-state evaluation slice: the model must learn a representation over the multibasin state space without being told basin labels, basin counts, or which states lie safely inside a basin. The deep-state slice is used only after training to score whether support families identify clean basin interiors. It is not used to train the encoder or transition, and it is not the forecasting evaluation population.

The saved best checkpoint is chosen by a fixed validation rule during training. Every 500 optimization steps, and again at the final step, the current model is rolled out for 200 steps from 16 held-out initial states using every-step re-encoding. The checkpoint with the lowest final validation error is saved as a checkpoint.

Table 4: Dysts forecasting systems. These systems are used only as external long-horizon forecasting stress tests.

System	State dimension
Chua	3
Dadras	3
Dequan Li	3
Hadley	3
Lu–Chen–Cheng	3
Qi–Chen	3
Sakarya	3
San–Um–Srisuchinwong	3
Shimizu–Morioka	3
Wang–Sun	3

The main Dysts setup serves as a forecasting stress test from an external benchmark of nontrivial systems, and focuses on LISTA, LISTA–BD, Sparse MLP, Sparse MLP–BD, and Dense MLP as the primary models for comparison. Dysts rows use latent dimension $d_z = 256$, 100,000 optimization steps, minibatches of 256 windows, and rollout length $L = 10$ windows. Dysts training trajectories are drawn from native trajectory caches with standardized coordinates, 200 cached trajectories, 30,000 stored observations per trajectory, and a 2,000-step warm-up. The current table reports the $dt\times30$ version of this benchmark: the stored Dysts observation interval is set to 30 times the intrinsic Dysts timestep for the corresponding system, and evaluation uses reduced horizon indices through $H = 5000$. Initial conditions are based on each Dysts system's default initial condition; the remaining cached trajectories perturb that state with Gaussian noise at scale 0.2 times the coordinate-wise Dysts standard deviation. Train, validation, and test caches are split so held-out trajectories are disjoint from training windows and reusable across model seeds. Long-horizon evaluation uses the disjoint held-out test cache, with a batch of 100 held-out test windows, equivalently rollout initial states, for each system and seed. LISTA rows use the same learning rates as the multibasin runs, while MLP rows use learning rates 10^{-4} for encoder/decoder parameters and 10^{-5} for the latent transition. Sparse models use $\lambda_{\text{sp}} = 6\times10^{-3}$, while the Dense MLP baseline sets $\lambda_{\text{sp}} = 0$. The soft-block diagonal LISTA model outperformed the dense MLP but was not included in the main table due to weaker statistical significance and worse relative performance than the rest of the LISTA models; we do not have a compelling explanation for why. Perhaps soft-block diagonal regularization provides the worst of both worlds in terms of Koopman transition structure for long horizon forecasting.

The Dysts systems have system-specific native integration intervals, so the cached sequences should be interpreted as benchmark observation sequences rather than as trajectories normalized to a common physical time. Coordinates are standardized before training and evaluation. The models never observe the vector field, the continuous solver, or the native integration substeps; they receive uniformly spaced stored states and are evaluated in stored-step units.

Training recipe selection. Earlier exploratory sweeps varied LISTA depth, shrinkage scale, sparsity coefficient, sign-split encodings, transition structure, and MLP sparsity controls. Those pilots used shorter budgets or broader system lists, so their numeric grids are not treated as final protocol here. The paper-facing appendix instead reports the fixed model rows, training budgets,

validation rule, held-out splits, and statistical units used for the final claims. In particular, staged recipe-discovery sweeps with 10,000-step, three-seed cells and one-step training windows were used only to choose candidate recipes. They are not pooled with the final 200,000-step, held-out-split evaluations and should not be interpreted as paper evidence.

Evaluation splits. All reported forecasting and interpretability results are evaluated on held-out trajectories or held-out cache windows, not on the training windows. Multibasin forecasting uses 100 held-out initial conditions for each system and seed and is evaluated on all held-out rollouts, not restricted to the deep-state slice. Routing fits local predictors on 256 disjoint held-out trajectories of length 256 and evaluates them on a separate set of 128 held-out rollouts. The Dysts long-horizon table uses 100 held-out cached test windows for each system and seed. Procedural benchmark annotations define evaluation slices, score support–basin agreement, construct controlled transfer pairs, and compute statistical summaries.

For support–basin diagnostics, the main manuscript uses the global deep-state slice. Given distances from an evaluation state to the closest and second-closest benchmark attractor centers, the basin-depth margin is the difference between those two distances. The global deep-state slice is the top quartile of this margin over all held-out evaluation states in a system. It removes boundary-adjacent and transition states for the static support-alignment question, but it can underrepresent shallower basins when attractor depths are unequal. The per-basin deep-slice robustness check instead takes the top quartile within each basin so every represented basin contributes relative-deep states.

The support-routed operator-fitting split contains $256 \times 256 = 65{,}536$ adjacent latent transitions per trained checkpoint and support definition. At the longest routed horizon reported in table 2a, $h = 1000$, the separate forecast split contains $128 \times 1000 = 128{,}000$ one-step target positions along the evaluated rollouts. By comparison, the multibasin training budget samples $200{,}000 \times 256 \times 8 = 409{,}600{,}000$ one-step transition targets from online rollout length $L = 8$ windows; this is a sampling-budget comparison rather than a fixed finite training-set size, because the procedural training windows are regenerated from the environment.

Periodic forecasting. All forecast horizons h and re-encoding periods m are reported in stored observation steps, not in equal physical time across systems. The no-reencoding rollout applies the learned latent transition for the full horizon before decoding. The periodic rollout decodes and re-encodes the model's own predicted state at a fixed period m. The multibasin forecasting tables select the best score over periods $m \in \{10, 25, 50, 100\}$. The Dysts $dt\times30$ evaluation uses periods $m \in \{10, 25, 50, 100, 150, 200\}$. These choices are fixed before aggregation and do not use the true future trajectory.

The fixed re-encoding grid is used only at evaluation time. Checkpoint selection uses the validation proxy described above, not a search over the final test-period grid, and the reported point estimates aggregate all completed seeds rather than selecting the best seed. The best-periodic score should therefore be read as a controlled diagnostic of whether a model benefits from regular support refresh at deployment, not as a training procedure or teacher-forced correction.

Support diagnostics. The wrong-support ablation first constructs, for each basin in a testable system, a canonical deep-slice S_{abs} mask from the most common exact mask observed in that basin. The ablated rollout replaces the inferred S_{abs} mask with a canonical mask from a different basin and holds that replacement fixed. Table 1 reports the wrong-support/base error ratio at horizons $h = 1$ and $h = 20$; larger ratios indicate stronger functional dependence on the selected support.

Routing details. Support-routed forecasting uses fixed-size S_{top8} masks so every encoded state receives an eight-index support key before family merging. The main table reports F_{top8}-local routing, which merges nearby top-8 masks into support families with Jaccard threshold 0.5 and fits centered affine local maps $T_c(z) = \bar{z}_c + K_c(z - \bar{z}_c)$. The ridge penalty is 10^{-4}. At evaluation time, a local route is used only if it was observed in at least 128 fitted transitions; otherwise the rollout falls back to the global latent transition. The reported routed/global ratio always compares the routed prediction with the same model's own global rollout, not with another model family. Exact gated S_{top8} and exact S_{top8}-local routes are retained in table 16 as route-form diagnostics rather than as the main paper claim.

The LISTA + local K_c row in table 1 is a second-stage forecasting variant rather than a separately trained representation. It freezes the Table 1 LISTA encoder and decoder, constructs F_{top8} support-family routes with Jaccard threshold 0.40, initializes each family-local transition map from the learned global K, and trains only those local transition maps for 50,000 rollout-loss optimization steps. Evaluation uses periodic decode/re-encode with period 5 and reroutes at each step. Because the encoder is frozen, support-alignment diagnostics such as $H(B \mid F_{\text{abs}})$ and $\overline{|F_{\text{abs}}|}$ are the same representation diagnostics as LISTA; Table 1 therefore reports this row as forecasting-only with inherited support diagnostics.

Support refresh. The support-refresh experiment uses exact S_{top8} supports and the two re-encoding periods reported in table 2b, $m \in \{1, 10\}$. Each comparison starts from a controlled source-to-target transfer with 32 pre-transition steps, 32 bridge steps, and 128 post-transition steps; the displayed benchmark rows score post-start continuation rollouts of horizon 64. The frozen-source rollout keeps using the source S_{top8} mask after target entry, while the refreshed-current rollout periodically decodes, re-encodes, and re-extracts S_{top8} from its own predicted state. Controlled basin labels are used only to construct and evaluate this benchmark transfer protocol. Benchmark annotations construct valid transfer pairs and score post-entry routing.

B Training-dynamics diagnostic

The main tables report held-out aggregate performance from validation-selected checkpoints. As a diagnostic for the optimization trajectory, figure 4 shows the metric histories for the six controlled multibasin model rows on one representative benchmark system, `gated_local_linear`, with seed 0. The curves use the same training protocol and validation rule described in section A: validation rollouts are evaluated every 500 optimization steps, and the saved checkpoint is the step with the lowest validation final error. The diagnostic is not used for model ranking; it is intended to show how the reported checkpoints arise along training.

The validation traces generally improve through training for the sparse rows, while the Dense MLP remains separated by a substantially larger validation error. The best validation checkpoints for the sparse rows occur late in training, between 179,500 and 193,500 steps in this trace, whereas the Dense MLP reaches its best validation value earlier. The spectral-radius panel shows that the learned transition remains close to the unit circle throughout training. This supports the qualitative reading that long-horizon behavior is tied to both validation error and the stability of the learned Koopman transition, while avoiding the stronger claim that $\rho(K)$ closest to one is always optimal.

For the external chaotic-flow benchmark, figure 5 gives the same diagnostic in aggregate form. Each curve uses seed 0 and summarizes the retained 10 Dysts $dt{\times}30$ systems by the median over finite per-system metric values at each logged step. We use a system median here rather than a single representative Dysts system because the individual chaotic flows have different scales and occasional

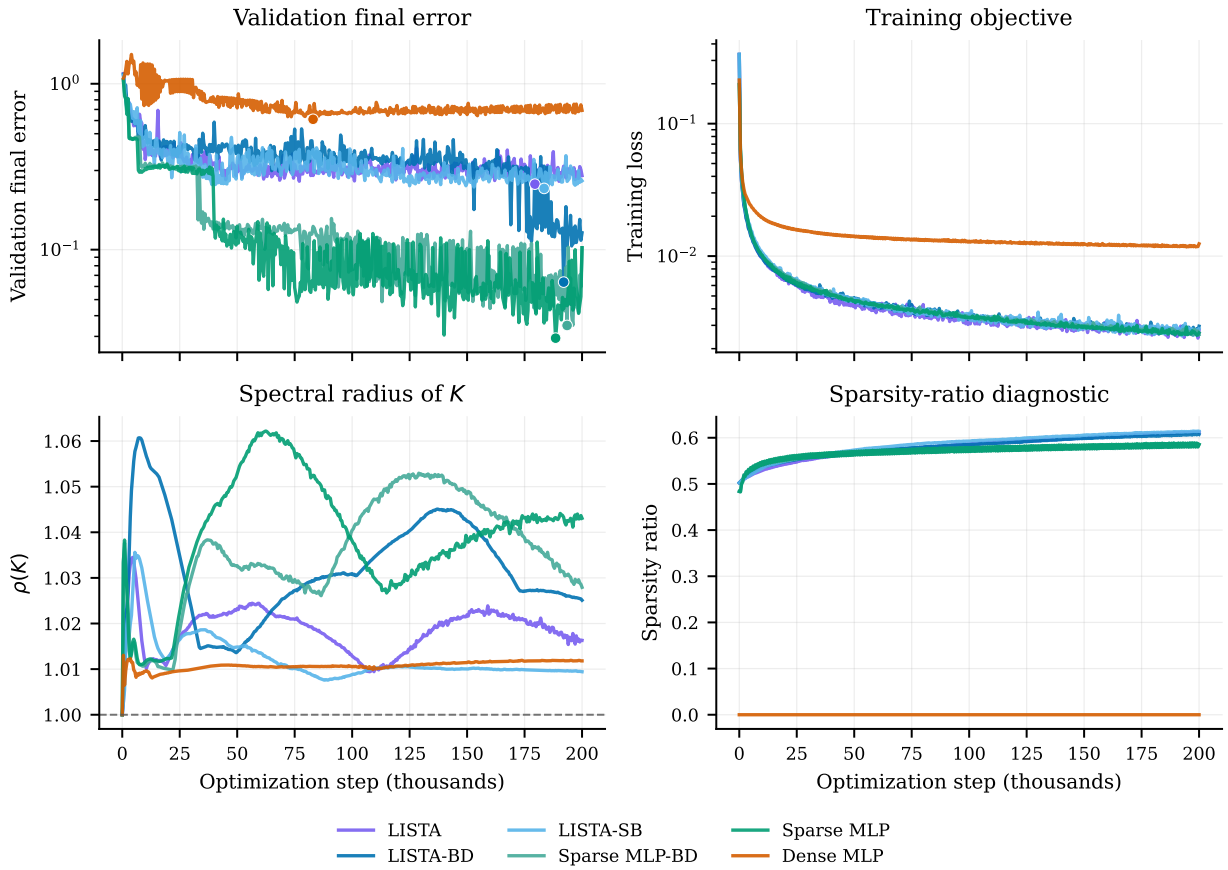


Figure 4: Representative training-dynamics diagnostic on `gated_local_linear`, seed 0, for the six controlled multibasin model rows. Validation final error is the held-out rollout error used by the checkpoint-selection rule; dots mark the minimum validation final error for each trace. The training objective, validation error, transition spectral radius $\rho(K)$, and sparsity-ratio diagnostic are plotted against optimization step. The horizontal dashed line marks $\rho(K) = 1$. These curves are a diagnostic of training progression and checkpoint selection, not an aggregate benchmark comparison.

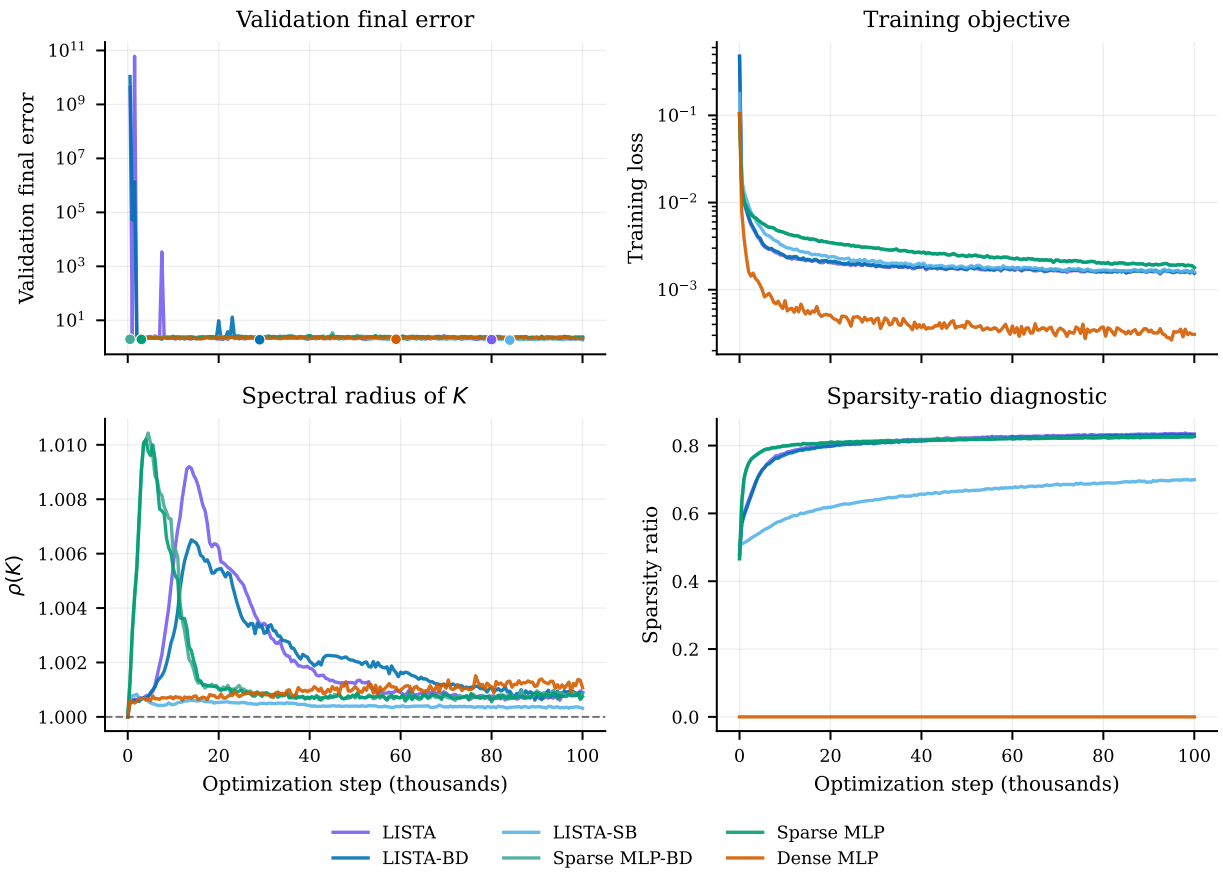


Figure 5: Dysts $dt\times30$ training-dynamics diagnostic. Curves summarize seed 0 over the retained 10 Dysts systems by the median over finite per-system metric values at each logged step. Validation final error is the same held-out rollout proxy used by checkpoint selection; dots mark the minimum median validation final error for each trace. The Dysts validation panel is noisier than the controlled trace because chaotic rollout errors can become very large early in training, so this figure is an appendix diagnostic of optimization and selection behavior rather than a main benchmark ranking.

very large finite validation rollout errors early in training. The resulting validation traces are therefore visibly noisier than the controlled-system trace, but they still show the checkpoint-selection path and the late-training behavior of the model families.

The Dysts diagnostic should be read more cautiously than the controlled benchmark diagnostic. The validation proxy can spike when a chaotic rollout leaves the attractor or compounds error early in training, and Dysts does not provide the controlled basin-label structure used by the main support-alignment claims. Still, the median traces are useful for checking that the retained $dt\times30$ runs are not selected from isolated early accidents: validation curves generally settle after the initial transient, and the learned transition spectra remain close to the unit circle by the end of training, with final median $\rho(K)$ values roughly between 1.0003 and 1.0011 across rows. As above, this supports the qualitative stability diagnostic without making $\rho(K)$ closest to one a sufficient criterion for forecasting quality.

Table 5: The 15 multibasin benchmark systems.

Display name	System key	Generator family	Basins
Local-linear gates	`gated_local_linear`	piecewise local-linear gates	3
Transfer-gated local-linear	`gated_transfer_linear`	piecewise transfer gates	3
Arrested spiral	`arrested_spiral`	spiral capture with Gaussian traps	5
Asymmetric three-well	`cal_asymmetric_3`	Gaussian wells with independent rotation	3
High-cross three-well	`cal_high_cross_3`	Gaussian wells with stronger rotation	3
Hexagonal six-well	`cal_hexagon_6`	Gaussian wells with independent rotation	6
Octagonal eight-well	`cal_octagon_8`	Gaussian wells with independent rotation	8
Pentagonal five-well	`cal_pentagon_5`	Gaussian wells with independent rotation	5
Square four-well	`cal_square_4`	Gaussian wells with independent rotation	4
Triple-well Duffing	`duffing_triple_well`	triple-well Duffing oscillator	3
SNIC multi-attractor	`snic_multi`	polar SNIC-like phase dynamics	3
Transition-routes four-well	`transition_routes_4`	route-augmented Gaussian wells	4
Depth-gradient four-well	`var_depth_gradient_4`	Gaussian wells with depth gradient	4
Diamond four-well	`var_diamond_4`	Gaussian wells on a diamond layout	4
L-shaped five-well	`var_l_shape_5`	Gaussian wells on an L-shaped layout	5

C Multibasin benchmark inventory and system definitions

Table 5 lists the 15 two-dimensional multibasin systems used in this paper. Every row contributes to the retained-system aggregate in table 1, table 2a, and figure 3a. Four systems are additionally displayed as the main benchmark visual panels in figure 1. The support-refresh subtable uses 12 retained systems for which the controlled-transfer construction produced eligible post-entry comparisons.

Basin labels are used only for benchmark annotation, controlled-transfer construction, and evaluation. Basin counts are benchmark metadata; for the structured-transition ablation rows, they are also used only to set a fixed architecture group count, as described in section A, and not for support extraction, support-family construction, routing, or rollout evaluation.

The 15 multibasin benchmark systems were generated procedurally with Claude Opus 4.5.

Stored-step convention. All retained systems are implemented as continuous-time vector fields $\dot{x} = f_s(x)$ and integrated with fourth-order Runge–Kutta to produce the stored observation map $F_{\Delta t}$ used in equation (1). The training windows contain only stored states x_k, not f_s, integration substeps, or basin assignments.

Gaussian-well family. Most retained catalog systems use a common two-dimensional Gaussian potential with an independent rotational drift. For $x = (x_1, x_2) \in \mathbb{R}^2$, let $Rx = (x_2, -x_1)$ and

$$V_s(x) = \sum_{i=1}^{M_s} -a_{s,i} \exp\left(-\frac{\|x - c_{s,i}\|_2^2}{2\sigma_{s,i}^2}\right) + \gamma_s(x_1^4 + x_2^4).$$

The implemented vector field is

$$\dot{x} = -\nabla V_s(x) + \omega_s R x. \tag{17}$$

Define $p_i^{(M)}(r, \varphi) = r(\cos(2\pi i/M + \varphi), \sin(2\pi i/M + \varphi))$, $i = 0, \ldots, M-1$. The retained Gaussian-well systems instantiate equation (17) with the parameters in table 6.

Table 6: Parameters for retained Gaussian-well systems. Repeated pairs in the (a_i, σ_i) column apply to every listed center.

System	$\{c_i\}$	(a_i, σ_i)	(ω, γ)
High-cross three-well	$\{p_i^{(3)}(1.8, \pi/2)\}$	$(3.0, 0.5)$	$(2.0, 0.03)$
Hexagonal six-well	$\{p_i^{(6)}(1.7, 0)\}$	$(2.0, 0.5)$	$(1.0, 0.03)$
Octagonal eight-well	$\{p_i^{(8)}(2.2, 0)\}$	$(3.0, 0.5)$	$(0.90, 0.02)$
Pentagonal five-well	$\{p_i^{(5)}(1.8, \pi/2)\}$	$(2.0, 0.5)$	$(1.1, 0.03)$
Square four-well	$\{p_i^{(4)}(1.8, \pi/4)\}$	$(3.0, 0.5)$	$(1.0, 0.03)$
Diamond four-well	$\{(0, 2.2), (2.2, 0), (0, -2.2), (-2.2, 0)\}$	$(2.5, 0.5)$	$(1.0, 0.02)$
L-shaped five-well	$\{(-1.5, 1.5), (-1.5, 0), (-1.5, -1.5), (0, -1.5), (1.5, -1.5)\}$	$(2.5, 0.5)$	$(1.0, 0.03)$
Asymmetric three-well	$\{(0, 1.8), (-1.5, -0.9), (1.5, -0.9)\}$	$\{(2.5, 0.55), (1.5, 0.4), (2.0, 0.5)\}$	$(1.0, 0.03)$
Depth-gradient four-well	$\{(-1.3, 1.3), (1.3, 1.3), (1.3, -1.3), (-1.3, -1.3)\}$	$\{(2.2, 0.55), (2.5, 0.5), (3.0, 0.5), (3.5, 0.5)\}$	$(1.3, 0.03)$

The transition-routes system uses the same Gaussian-well form with centers

$$\{c_i\} = \{(-1.8, 1.8), (1.8, 1.8), (-1.8, -1.8), (1.8, -1.8)\},$$

$a_i = 3.0$, $\sigma_i = 0.6$, $\omega = 1.0$, and $\gamma = 0.03$, and adds a corridor-dependent rotation term:

$$\dot{x} = -\nabla V_s(x) + \big(1.0 + 0.3\, C_{\text{route}}(x_2)\big) Rx, \qquad C_{\text{route}}(x_2) = e^{-(x_2-1.8)^2/0.3} + e^{-(x_2+1.8)^2/0.3}. \quad (18)$$

Native gated local-linear systems. Both native gated systems place three centers on a circular layout, but with different radii and region definitions. Let $Q(\alpha) = \left(\begin{smallmatrix} \cos\alpha & -\sin\alpha \\ \sin\alpha & \cos\alpha \end{smallmatrix}\right)$ and $\alpha_b = 2\pi b/3$, $b \in \{0, 1, 2\}$.

For `gated_local_linear`, $c_b = 1.75(\cos\alpha_b, \sin\alpha_b)$. Inside the radius-1.05 core of basin b,

$$\dot{x} = A_b(x - c_b), \qquad A_b = Q(\alpha_b)\widetilde{A}_b Q(\alpha_b)^\top,$$

with

$$\widetilde{A}_0 = \left(\begin{smallmatrix} -0.9 & -1.2 \\ 1.2 & -0.9 \end{smallmatrix}\right), \quad \widetilde{A}_1 = \left(\begin{smallmatrix} -1.35 & 0.2 \\ -0.3 & -0.7 \end{smallmatrix}\right), \quad \widetilde{A}_2 = \left(\begin{smallmatrix} -0.7 & -0.1 \\ 0.5 & -1.2 \end{smallmatrix}\right).$$

Outside the cores, the active sector is the nearest angular sector $s(x)$ and the shared gate matrix

$$G = \left(\begin{smallmatrix} -1.35 & -0.9 \\ 0.9 & -1.35 \end{smallmatrix}\right)$$

drives the state as $\dot{x} = G(x - c_{s(x)})$.

For `gated_transfer_linear`, $c_b = 1.85(\cos\alpha_b, \sin\alpha_b)$. The implemented radii and widths are

$$\rho_{\text{core}} = 0.30, \quad \rho_{\text{source}} = 0.80, \quad \rho_{\text{exit}} = 0.60, \quad \beta_{\text{exit}} = 0.72,$$
$$w_{\text{chan}} = 0.22, \quad \delta_{\text{lane}} = 0.28, \quad \rho_{\text{hand}} = 0.45.$$

The core matrices use the same rotated form with templates

$$\left(\begin{smallmatrix} -1.0 & -1.1 \\ 1.1 & -1.0 \end{smallmatrix}\right), \quad \left(\begin{smallmatrix} -1.4 & 0.2 \\ -0.2 & -0.7 \end{smallmatrix}\right), \quad \left(\begin{smallmatrix} -0.8 & -0.3 \\ 0.5 & -1.3 \end{smallmatrix}\right).$$

For each ordered pair $p = (s, t)$, $s \neq t$, define

$$d_{s\to t} = \frac{c_t - c_s}{\|c_t - c_s\|_2}, \qquad n_{s\to t} = (-d_{s\to t,2}, d_{s\to t,1}), \qquad o_t = \frac{c_t}{\|c_t\|_2},$$

and $\chi_{s,t} = 1$ when $\sin(\alpha_s - \alpha_t) \geq 0$, otherwise $\chi_{s,t} = -1$. The implemented channel entry and handoff points are

$$e_p = c_s + \rho_{\text{source}} d_{s\to t} + \chi_{s,t}\delta_{\text{lane}} n_{s\to t},$$
$$h_p = c_t + \rho_{\text{hand}} o_t + \chi_{s,t}\delta_{\text{lane}}(-o_{t,2}, o_{t,1}),$$

with channel direction $d_p = (h_p - e_p)/\|h_p - e_p\|_2$, channel normal $n_p = (-d_{p,2}, d_{p,1})$, and channel length $\ell_p = (h_p - e_p) \cdot d_p$.

The vector field is piecewise analytic. Core regions $\|x - c_b\|_2 \le \rho_{\text{core}}$ use $A_b(x - c_b)$. Source annuli use $0.65A_b(x - c_b)$, and other non-channel background regions use $0.50A_b(x - c_b)$ for the nearest center c_b. Exit wedges for $p = (s, t)$ are source-neighborhood states outside the core with $\|x - c_s\|_2 \ge \rho_{\text{exit}}$ and $(x - c_s) \cdot d_{s\to t} \ge \|x - c_s\|_2 \cos\beta_{\text{exit}}$; they use

$$\dot{x} = v_{\text{exit}} d_{s\to t} - \lambda_{\text{exit}}\big((x - c_s) \cdot n_{s\to t}\big) n_{s\to t}, \qquad v_{\text{exit}} = 1.0, \quad \lambda_{\text{exit}} = 1.8;$$

and channel rectangles satisfying

$$0 \le (x - e_p) \cdot d_p \le \ell_p, \qquad |(x - e_p) \cdot n_p| \le w_{\text{chan}}$$

use

$$\dot{x} = v_{\text{chan}} d_p - \lambda_{\text{chan}}\big((x - e_p) \cdot n_p\big) n_p, \qquad v_{\text{chan}} = 1.55, \quad \lambda_{\text{chan}} = 2.8.$$

Other specialized systems. The arrested spiral system has a background spiral flow plus four Gaussian traps and an origin trap:

$$\dot{x} = -0.3x + 2.0Rx - 4.0 \sum_{i=1}^{4} e^{-\|x - c_i\|_2^2/(2\cdot 0.25)} \frac{x - c_i}{0.25} - 1.5e^{-\|x\|_2^2/0.3} \frac{x}{0.15} - 0.02(x_1^3, x_2^3),$$

with $c_i \in \{(1.5, 0), (0, 1.8), (-1, -1), (0.8, -1.5)\}$. Its fifth basin is the origin trap, matching the benchmark manifest count.

The triple-well Duffing system uses $x = (q, p)$ and

$$V(q) = q^6/6 - q^4/2 + aq^2, \qquad V'(q) = q^5 - 2q^3 + 2aq,$$

with $a = 0.3$, damping $\delta = 0.5$, rotation strength $\omega = 1.0$, and soft cubic confinement $-\epsilon u^3/s^3$ with $\epsilon = 0.003$ and $s = 4.0$:

$$\begin{aligned} \dot{q} &= (1 + \omega)p - \epsilon q^3/s^3, \\ \dot{p} &= -V'(q) - \delta p - \omega q - \epsilon p^3/s^3. \end{aligned} \tag{19}$$

The SNIC system is implemented in polar form and then converted to Cartesian coordinates. With $\varepsilon_{\text{num}} = 10^{-8}$, $r^2 = x_1^2 + x_2^2 + \varepsilon_{\text{num}}$, $r = \sqrt{r^2}$, $\theta = \text{atan2}(x_2, x_1)$, $c_\theta = x_1/(r + \varepsilon_{\text{num}})$, and $s_\theta = x_2/(r + \varepsilon_{\text{num}})$,

$$\dot{r} = r(1 - r^2) - 0.3\cos(3\theta), \qquad \dot{\theta} = 1 - 1.2\cos(3\theta).$$

The Cartesian vector field is

$$\begin{aligned} \dot{x}_1 &= \dot{r}c_\theta - r\dot{\theta}s_\theta - 0.01x_1(x_1^2 + x_2^2) + 0.5x_2, \\ \dot{x}_2 &= \dot{r}s_\theta + r\dot{\theta}c_\theta - 0.01x_2(x_1^2 + x_2^2) - 0.5x_1. \end{aligned}$$

These analytic generators define the benchmark trajectories. Held-out evaluation annotations are produced by the benchmark label helpers or by endpoint-rollout basin labeling, and the learned models observe only stored states.

Table 7: The ten Dysts systems used in the $dt\times30$ forecasting benchmark. The "stored step" column is 30 times the native Dysts integration interval.

Display name	System key	Dimension	Native dt	Stored step $30dt$
Chua	`Chua`	3	$2.8474745791\times10^{-4}$	$8.5424237373\times10^{-3}$
Dadras	`Dadras`	3	$6.5782963827\times10^{-4}$	$1.9734889148\times10^{-2}$
Dequan Li	`DequanLi`	3	$1.6763993999\times10^{-5}$	$5.0291981997\times10^{-4}$
Hadley	`Hadley`	3	$2.9086847808\times10^{-4}$	$8.7260543424\times10^{-3}$
Lu–Chen–Cheng	`LuChenCheng`	3	$1.8469678280\times10^{-4}$	$5.5409034839\times10^{-3}$
Qi–Chen	`QiChen`	3	$7.8371061844\times10^{-5}$	$2.3511318553\times10^{-3}$
Sakarya	`Sakarya`	3	$9.9704617436\times10^{-4}$	$2.9911385231\times10^{-2}$
San–Um–Srisuchinwong	`SanUmSrisuchinwong`	3	$1.4933288881\times10^{-3}$	$4.4799866644\times10^{-2}$
Shimizu–Morioka	`ShimizuMorioka`	3	$2.4080013336\times10^{-3}$	$7.2240040007\times10^{-2}$
Wang–Sun	`WangSun`	3	$5.3924987498\times10^{-3}$	$1.6177496249\times10^{-1}$

D Dysts benchmark inventory and system definitions

This appendix lists the ten Dysts systems used in the paper-facing $dt\times30$ forecasting benchmark. The systems are drawn from Dysts (Gilpin, 2021, 2023). We used the continuous-time right-hand sides and metadata from the pinned package version resolved in the project environment, `dysts` 0.96. All ten systems are three-dimensional autonomous flows. The equations below are written in unstandardized native Dysts coordinates (x, y, z); the training cache standardizes coordinates only after trajectories are generated.

Closed-form convention. For each system, the stored observations are generated from $\dot{x} = f_s(x)$ with observation interval $\Delta t = 30\, dt_{\text{native}}$, where dt_{native} is the system-specific Dysts integration interval in table 7. The learner observes only the resulting stored states, not the vector field, continuous-time solver, or substeps. All ten systems have explicit closed-form right-hand sides in Dysts. Most are polynomial vector fields. The Chua and San–Um–Srisuchinwong systems include absolute-value terms, so they are piecewise smooth rather than globally analytic at the corresponding switching surfaces.

Chua. The Chua system uses the piecewise-linear diode nonlinearity

$$h(x) = m_1 x + \frac{1}{2}(m_0 - m_1)\big(|x+1| - |x-1|\big),$$

with parameters

$$\alpha = 15.6, \qquad \beta = 28, \qquad m_0 = -1.142857, \qquad m_1 = -0.71429.$$

The vector field is

$$\begin{aligned} \dot{x} &= \alpha\big(y - x - h(x)\big), \\ \dot{y} &= x - y + z, \\ \dot{z} &= -\beta y. \end{aligned} \tag{20}$$

Dadras. With parameters

$$c = 2, \qquad e = 9, \qquad o = 2.7, \qquad p = 3, \qquad r = 1.7,$$

the Dadras system is

$$\begin{aligned}\dot{x} &= y - px + oyz,\\ \dot{y} &= ry - xz + z,\\ \dot{z} &= cxy - ez.\end{aligned} \tag{21}$$

Dequan Li. With parameters

$$a = 40, \qquad c = 1.833, \qquad d = 0.16, \qquad \varepsilon = 0.65, \qquad f = 20, \qquad k = 55,$$

the Dequan Li system is

$$\begin{aligned}\dot{x} &= a(y - x) + dxz,\\ \dot{y} &= kx + fy - xz,\\ \dot{z} &= cz + xy - \varepsilon x^2.\end{aligned} \tag{22}$$

Hadley. With parameters

$$a = 0.2, \qquad b = 4, \qquad f = 9, \qquad g = 1,$$

the Hadley system is

$$\begin{aligned}\dot{x} &= -y^2 - z^2 - ax + af,\\ \dot{y} &= xy - bxz - y + g,\\ \dot{z} &= bxy + xz - z.\end{aligned} \tag{23}$$

Lu–Chen–Cheng. With parameters

$$a = -10, \qquad b = -4, \qquad c = 18.1,$$

the Lu–Chen–Cheng system is

$$\begin{aligned}\dot{x} &= -\frac{ab}{a+b}x - yz + c,\\ \dot{y} &= ay + xz,\\ \dot{z} &= bz + xy.\end{aligned} \tag{24}$$

Qi–Chen. With parameters

$$a = 38, \qquad b = 2.666, \qquad c = 80,$$

the Qi–Chen system is

$$\begin{aligned}\dot{x} &= a(y - x) + yz,\\ \dot{y} &= cx + y - xz,\\ \dot{z} &= xy - bz.\end{aligned} \tag{25}$$

Sakarya. With parameters

$$a = -1, \qquad b = 1, \qquad c = 1, \qquad h = 1, \qquad p = 1, \qquad q = 0.4, \qquad r = 0.3, \qquad s = 1,$$

the Sakarya system is

$$\begin{aligned}\dot{x} &= ax + hy + syz,\\ \dot{y} &= -by - px + qxz,\\ \dot{z} &= cz - rxy.\end{aligned} \tag{26}$$

San–Um–Srisuchinwong. With parameter $a = 2$, the San–Um–Srisuchinwong system is

$$\begin{aligned} \dot{x} &= y - x, \\ \dot{y} &= -z \tanh(x), \\ \dot{z} &= -a + xy + |y|. \end{aligned} \tag{27}$$

Shimizu–Morioka. With parameters

$$a = 0.85, \qquad b = 0.5,$$

the Shimizu–Morioka system is

$$\begin{aligned} \dot{x} &= y, \\ \dot{y} &= x - ay - xz, \\ \dot{z} &= -bz + x^2. \end{aligned} \tag{28}$$

Wang–Sun. With parameters

$$a = 0.2, \qquad b = -0.01, \qquad d = -0.4, \qquad e = -1, \qquad f = -1, \qquad q = 1,$$

the Wang–Sun system is

$$\begin{aligned} \dot{x} &= ax + qyz, \\ \dot{y} &= bx + dy - xz, \\ \dot{z} &= ez + fxy. \end{aligned} \tag{29}$$

E Support definitions and sensitivity

Section 3.4 defines the notation used in the main text. This appendix records the implementation-level objects behind that notation and explains how to read the corresponding diagnostics. All support objects are computed after training from the learned encoder output $z = E_\theta(x)$. Basin labels, basin counts, and trajectory-to-basin assignments are not used to train the model, define a support, or build a support family; when labels appear below they are evaluation-only quantities used to score or display the learned objects.

From a latent vector to an exact support key. The reducers first convert each latent vector into a Boolean mask $m(x) \in \{0,1\}^{d_z}$. The exact support $S(x)$ is the index set of the true entries of this mask, but the code compares exact supports by serializing the Boolean mask itself: two states have the same exact support if and only if all d_z Boolean entries match. The three support rules used in the paper are

$$m_{\mathrm{abs},i}(x) = \mathbf{1}\{|z_i| > 10^{-3}\}, \qquad S_{\mathrm{abs}}(x) = \{i : m_{\mathrm{abs},i}(x) = 1\}, \tag{30}$$

$$m_{\mathrm{top8},i}(x) = \mathbf{1}\{i \in \mathrm{top}_8(|z|)\}, \qquad S_{\mathrm{top8}}(x) = \{i : m_{\mathrm{top8},i}(x) = 1\}, \tag{31}$$

$$m_{\mathrm{rel},i}(x) = \mathbf{1}\{|z_i| > 0.1 \max(\max_j |z_j|, \epsilon)\}, \qquad S_{\mathrm{rel}}(x) = \{i : m_{\mathrm{rel},i}(x) = 1\}, \tag{32}$$

with $\epsilon = 10^{-12}$ in the implementation. The inequalities are strict. The epsilon only affects the degenerate all-zero relative-threshold case, where S_{rel} is empty. S_{abs} is a magnitude-thresholded active set, so $|S_{\mathrm{abs}}|$ is a measured state-dependent statistic rather than a fixed sparsity budget. S_{top8} instead always begins from eight active coordinates when $d_z \geq 8$; it ignores absolute scale and gives every state a fixed-size exact key before family merging.

From exact supports to support families. For a chosen support rule and evaluation collection, the implementation counts distinct exact support keys, sorts them from most to least frequent, and uses the serialized key as a deterministic tie-breaker. It then performs the leader-style greedy Jaccard merge described in section 3.4. Each family stores an actual Boolean support vector as its representative. A new exact mask joins the existing representative with the largest Jaccard similarity if that similarity is at least 0.5; otherwise it starts a new family. Representatives are not optimized, averaged, or updated as centroids. Family labels are therefore run- and collection-specific integer identifiers, and only their induced partition of states is meaningful.

Support-family creation algorithm. For completeness, the family construction used for both F_{abs} and F_{top8} is:

> **Input:** evaluation states $\mathcal{X}$, a support rule q, and Jaccard threshold $\tau = 0.5$.
> **Output:** family label $F_q(x)$ for each $x \in \mathcal{X}$, and family representatives $\{r_f\}$.
>
> 1. Compute one exact Boolean mask $m_q(x) \in \{0,1\}^{d_z}$ for every $x \in \mathcal{X}$.
> 2. Count the multiplicity $c(m)$ of each distinct exact mask m.
> 3. Sort the distinct masks in decreasing $c(m)$, using the serialized Boolean mask as a deterministic tie-breaker.
> 4. Initialize an empty list of families and representatives.
> 5. For each distinct mask m in the sorted order:
>   a. If no family exists, create a new family f, set $r_f \leftarrow m$, and assign m to f.
>   b. Otherwise compute $f^\star = \arg\max_f J(m, r_f)$ over existing family representatives.
>   c. If $J(m, r_{f^\star}) \geq \tau$, assign every occurrence of m to family $f^\star$.
>   d. If $J(m, r_{f^\star}) < \tau$, create a new family f, set $r_f \leftarrow m$, and assign every occurrence of m to f.
> 6. For each state x, return $F_q(x)$, the family assigned to its exact mask $m_q(x)$.

The representative update $r_f \leftarrow m$ occurs only when a new family is created. Existing representatives are never averaged, recomputed as majority masks, or moved toward later assigned masks.

For top-eight masks, $J(m, r) \geq 0.5$ is equivalent to at least six shared active coordinates because both masks have size eight. For S_{abs}, the same threshold is adaptive to mask size through the intersection-over-union denominator. A lower Jaccard threshold merges more exact supports and can collapse distinct basins; a higher threshold separates more masks and can make $H(B \mid F)$ small by over-fragmenting each basin. We therefore fix 0.5 rather than tuning it post hoc and interpret any entropy result together with the corresponding family count.

Which support object is used where.

S_{abs}.
This is the strict active-coordinate object. It is used to build F_{abs}, to report exact-support diagnostics such as $H(B \mid S_{\text{abs}})$, $H(S_{\text{abs}} \mid B)$, exact-support uniqueness, and $|S_{\text{abs}}|$, and to form canonical masks for wrong-support interventions. In the intervention code, the canonical mask for basin b is the most frequent S_{abs} key among candidate deep-slice states from basin b. The "wrong" condition replaces it with canonical masks from other represented basins and holds that mask fixed during the masked latent rollout.

F_{abs}.
This is the main basin-support alignment object in table 1. It asks whether the many exact S_{abs}

masks produced by a sparse encoder compress into basin-scale families. The primary directional metric is $H(B \mid F_{\text{abs}})$: low values mean that knowing the support family leaves little uncertainty about the withheld basin label. The count $\overline{|F_{\text{abs}}|}$ is deliberately non-directional and should be compared with the number of basin labels represented in the evaluated slice.

S_{top8}.
This exact fixed-size support is used for periodic support refresh and for appendix route-form ablations. In the refresh experiment, re-encoding changes z, and S_{top8} is re-selected from that current latent vector; the reported target-route fraction is computed only after mapping support classes to basins for evaluation.

F_{top8}.
This is the main label-free routing object in table 2a. Local affine predictors are fit on a disjoint fitting set by grouping transitions according to the current state's F_{top8} label. During rollout, the route is recomputed from the current predicted latent state. If the exact key was seen during fitting, its stored family is used; otherwise it is assigned to the nearest stored family prototype when the Jaccard similarity reaches 0.5. If no fitted local map is available, the evaluator falls back to the same model's global Koopman step. Figure 6 visualizes representative F_{top8} masks by plotting their active coordinate indices.

S_{rel}.
This scale-relative support removes global latent-amplitude effects and is useful as a sensitivity rule for centered local-law diagnostics. It is not the main routing object because exact relative-threshold supports fragment the route space more strongly than S_{top8}.

How to read the diagnostics. A low $H(B \mid S_{\text{abs}})$ or $H(B \mid F_{\text{abs}})$ means that the support object predicts basin identity on the evaluated benchmark states. It does not by itself imply a compact one-support-per-basin code: a model can achieve low $H(B \mid S_{\text{abs}})$ while assigning many different exact masks to the same basin. $H(S_{\text{abs}} \mid B)$, exact-support uniqueness, and $|S_{\text{abs}}|$ diagnose this fragmentation, while $H(B \mid F_{\text{abs}})$ and $\overline{|F_{\text{abs}}|}$ ask whether the fragmented exact masks merge into a basin-scale family structure.

The slice used for this diagnostic matters. The deep-state slice used in the main table is the global top quartile of the basin-depth margin, where the margin is the distance to the second-nearest benchmark attractor center minus the distance to the nearest attractor center. It removes boundary-adjacent states and is the cleanest test of basin-interior structure. On this slice, F_{abs} family counts are closest to the number of represented basins for the LISTA-style rows, with LISTA-BD giving $\overline{|F_{\text{abs}}|} = 3.03$ against a represented-basin mean of 3.00. On the all-state slice, which includes boundary and transition states, the closest count match at the fixed $J = 0.5$ threshold instead comes from the sparse MLP controls, but those count matches have substantially higher basin uncertainty than the LISTA rows. Whole-slice count matching should therefore be read as a compression diagnostic, not as the primary evidence for basin-support alignment.

The takeaways are as follows. F_{abs} is the paper-facing object for the claim that sparse supports identify basin interiors. S_{abs} is the intervention object for testing whether the active coordinates are functionally used. F_{top8} is the deployment-facing route object because it provides a fixed-size support key and can be recomputed from predicted latents without oracle labels. S_{rel} is a sensitivity object, not a headline support definition.

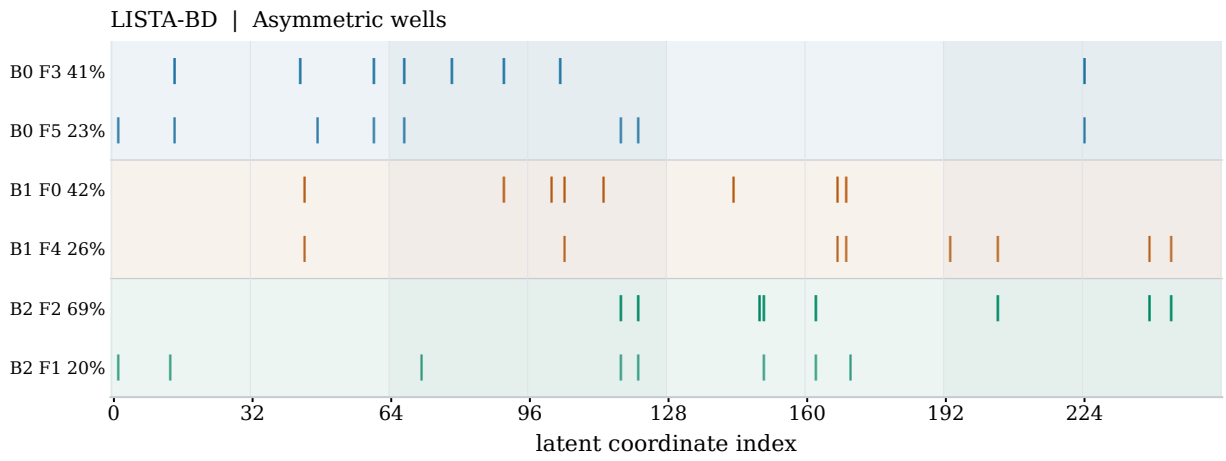


Figure 6: F_{top8} support-family active indices for the LISTA-BD KAE on the three-basin asymmetric wells system. Each row is one displayed F_{top8} family prototype from the seed-0 model; thin ticks mark the actual active latent coordinate indices in the prototype top-8 mask on the x-axis. Row labels give the post-hoc dominant evaluation basin, support-family id, and within-basin coverage. The basin names in the labels are evaluation-only annotations and are not used to construct the support families.

F Statistical testing protocol

The point estimates and significance annotations answer different questions. In the forecasting and support-diagnostic tables, point estimates usually summarize completed random training seeds within each benchmark system by an interquartile mean (IQM), then average those system summaries across systems. The main exception is the descriptive family-count column $\overline{|F_{\text{abs}}|}$, which averages per-system seed means because it is a count diagnostic rather than a directional performance metric. Routing and support-refresh ratio summaries use the matched log-ratio summaries described below. The statistical annotations ask whether the paired effect is reproducible under the experimental unit matched by the design. We therefore do not pool seeds, trajectories, transfers, and systems into one sample. A seed, controlled transfer instance, or benchmark system is used as the replicate only when that is the unit on which the two conditions are paired.

Paired effects and alternatives. For multibasin forecasting comparisons in table 1, $\text{MSE}_{s,r,h}$ is the held-out best-periodic MSE at rollout horizon h. Against the dense-latent MLP baseline (referred to as Dense), the paired effect for system s, seed r, and horizon h is

$$\delta_{s,r,h} = \log_{10} \text{MSE}^{\text{cand}}_{s,r,h} - \log_{10} \text{MSE}^{\text{Dense}}_{s,r,h} = \log_{10}\left(\text{MSE}^{\text{cand}}_{s,r,h} \,/\, \text{MSE}^{\text{Dense}}_{s,r,h}\right). \tag{33}$$

The one-sided alternative is $\delta < 0$, matching the table direction that lower MSE is better. The same log-ratio convention is used for F_{top8} routed/global comparisons in table 2a and refreshed/stale-source S_{top8} comparisons in table 2b. Log ratios are used because MSEs are positive, the observed errors are heavy-tailed across seeds, and the scientific question is usually multiplicative: whether one paired condition reduces error relative to the matched alternative. Tables display ratios or MSEs on the original scale, but the paired tests are performed on the corresponding log differences when the metric is an MSE ratio.

The support-alignment diagnostics use the same support notation as sections 3.4 and E: S_{abs} is the absolute-threshold active-coordinate set, F_{abs} is the greedy Jaccard family of those exact supports, S_{top8} is the eight largest-magnitude coordinates, and F_{top8} is the corresponding top-8 family. For $H(B \mid F_{\text{abs}})$ and the appendix $|S_{\text{abs}}|$ active-coordinate diagnostic, the paired difference is candidate minus Dense MLP with one-sided alternative candidate $<$ Dense. For wrong-support/base ratios, the alternative is candidate $>$ Dense, because a larger wrong-support/base ratio means that replacing the active S_{abs} mask with a different basin's canonical mask is more damaging. The displayed $\overline{|F_{\text{abs}}|}$ family count is intentionally descriptive rather than assigned a one-sided test: its interpretation is closeness to the number of represented deep-slice basins, not monotone increase or decrease.

Within-system brackets. Bracketed K/N counts in tables 1 and 2b are within-system reproducibility counts. For multibasin forecasting, $H(B \mid F_{\text{abs}})$, appendix $|S_{\text{abs}}|$, and wrong-support ablations, the paired unit is the training seed r: candidate and Dense MLP values are matched by the same system and seed. For support refresh, the paired unit is the controlled source-to-target transfer instance within a system; all completed training seeds contribute transfer instances to that within-system test.

Within each system and cell, we apply a one-sided paired Wilcoxon signed-rank test to the paired deltas in the prespecified direction. This is the appropriate within-system test because the observations are paired by construction, the per-system sample sizes are modest, and log-MSE or ratio deltas are not well described by a Gaussian paired-error model. The signed-rank test uses the magnitudes and signs of matched differences for a paired location-shift question without imposing a Gaussian error model; the one-sided alternative is fixed by the table arrow before looking at the cell. For each model–metric–horizon-or-period–subset cell, valid per-system p-values are Holm-corrected across the eligible systems for that cell at $\alpha = 0.05$. The reported K/N is the number of systems whose Holm-corrected p-value is below 0.05 over the number of systems with an eligible, non-degenerate paired test for that diagnostic.

This denominator convention explains the apparent differences across columns. Forecasting and the appendix active-count diagnostics have the full 15-system denominator when all systems have enough finite paired seeds. $H(B \mid F_{\text{abs}})$ can have a smaller denominator, currently 11 or 12 depending on the model row, because several systems have degenerate paired entropy deltas on the main deep-state slice. Wrong-support ablations have $N = 11$ because four systems have only one represented basin in the global top-quartile deep slice, so substituting a different basin's canonical S_{abs} mask is undefined. Support refresh has $N = 12$, the number of systems with valid controlled transfer coverage.

System-level support-based routing. The routing stars in table 2a are the main exception to the bracket-count rule. The displayed routing claim is a cross-system claim about whether F_{top8}-local predictors improve over the same model's global K rollout on the benchmark. Seeds are nested inside systems, so treating seed-level routed/global ratios from the same dynamical system as independent benchmark systems would overstate the sample size.

For each system, we first compute finite seed-level log ratios

$$\log_{10}\left(\text{MSE}^{\text{routed}}_{s,r,h} \,/\, \text{MSE}^{\text{global}}_{s,r,h}\right)$$

for the F_{top8}-local route and summarize them by a within-system IQM, denoted $D_{s,h}$. The table reports $10^{\text{mean}_s\, D_{s,h}}$, so values below one indicate lower routed MSE. Significance is tested with a one-sided exact sign-flip test on the finite system-level effects $D_{s,h}$, using the benchmark system as

the independent unit and the alternative $D_{s,h} < 0$. The sign-flip test is appropriate here because, under the paired null, each system-level effect can be assigned either sign while retaining its observed magnitude; the test therefore matches the paired cross-system claim and, with only 15 systems, avoids a normal approximation. The resulting p-values are Holm-corrected across the displayed model–horizon routing cells. The companion "system wins" column is a directional diagnostic from the censored per-system comparison, not the confirmatory p-value. In that diagnostic, systems with finite routed and global rollouts use the same log-ratio direction; a finite routed rollout paired with an invalid same-model global rollout is counted as a routed win, the reverse as a loss, and missing or both-invalid seed slots are neutral before the per-system direction is summarized.

Support-refresh counts. Table 2b reports exact S_{top8} refresh after controlled source-to-target basin transfer. The target-route fraction is descriptive: it measures how often the periodically re-encoded rollout selects the target-basin support class under the evaluation-only support-to-basin mapping. The significance count is attached only to the refreshed/stale-source support MSE ratio. For each valid controlled transfer instance u in system s, the paired effect is

$$\rho_{s,u,m} = \log_{10}\left(\text{MSE}^{\text{refreshed}}_{s,u,m} \,/\, \text{MSE}^{\text{stale}}_{s,u,m}\right),$$

where m is the re-encoding period and "stale" denotes the rollout that keeps using the source-basin S_{top8} mask after target entry. The one-sided alternative is $\rho_{s,u,m} < 0$, because lower refreshed/stale MSE means that re-encoding and re-selecting S_{top8} helps after transfer. Within each system and period, the paired Wilcoxon signed-rank test uses all valid controlled transfer instances pooled over completed seeds, and Holm correction is applied across the 12 eligible systems for that model-period cell.

Dysts forecasting table. The Dysts table in table 3 also uses systems, not seeds, as the independent units because the claim is whether a model improves over Dense MLP across benchmark systems. For each Dysts system and horizon, seeds are summarized within system as

$$I^{\text{cand}}_{s,h} = \text{IQM}_r\left\{\text{MSE}^{\text{cand}}_{s,r,h}\right\}, \qquad I^{\text{Dense}}_{s,h} = \text{IQM}_r\left\{\text{MSE}^{\text{Dense}}_{s,r,h}\right\}.$$

Each non-Dense row then contributes one paired system effect

$$d_{s,h} = \log_{10} I^{\text{cand}}_{s,h} - \log_{10} I^{\text{Dense}}_{s,h}.$$

The 10 Dysts systems are tested with a one-sided Wilcoxon signed-rank test with alternative $d_{s,h} < 0$. This is the appropriate unit because the Dysts claim is a benchmark-family forecasting claim, and seed variation has already been summarized inside each system. The test remains paired because each candidate system summary is compared with the Dense MLP summary on the same Dysts system and horizon. These p-values are Holm-corrected across all non-Dense model–horizon comparisons in the Dysts table, and the corrected values determine the displayed superscripts.

G Falsification and partition controls

The partition-control experiments are falsification checks for the support-routing interpretation in table 2a. They separate four questions. First, do post-hoc route-local centered affine maps $T_c(z) = \bar{z}_c + K_c(z - \bar{z}_c)$ improve over the same model's autonomous global-K rollout at all? Second, can the model-produced F_{top8} support family select those maps without basin labels or a known

basin count? Third, would an oracle basin partition, a generic latent-space partition, or an arbitrary count-matched partition explain the same effect? Fourth, does improvement over the same model's autonomous global-K rollout imply that the post-hoc routed predictor is also better than the best-periodic re-encoding forecaster used in the main forecasting tables?

All rows in table 8 use the current $d_z = 256$ artifacts and freeze the trained encoder E_θ, decoder D_ϕ, and learned global transition K. The route-operator fitting split contains 256 held-out trajectories with 256 adjacent encoded transitions each. For each selector, the current latent states define route classes c; for every class with at least 128 fitting transitions, we fit the centered ridge slope K_c in equation (14), with $\eta = 10^{-4}$, and roll out the affine map T_c in equation (15). Missing or underpopulated routes fall back to the same model's global K. The forecast split is disjoint from the fitting split, contains 128 held-out rollouts, and recomputes the route from the model's current predicted latent state during rollout. Thus the table changes only the route selector while holding the trained representation, local-map fitting rule, fallback rule, and global baseline fixed.

The selectors differ only in how they define the route class c_t. The support-family selector uses $c_t = F_{\text{top8}}(z_t)$, constructed by greedy Jaccard merging of exact S_{top8} masks at threshold 0.5, and uses no basin labels or basin count. The oracle-basin selector uses benchmark basin labels on the fitting trajectories and, during rollout, assigns the decoded current prediction $D_\phi(z_t)$ to the nearest evaluation basin center used by the labeling routine. This is an evaluation-only control: it uses information that is intentionally unavailable in the training/deployment setting. The latent-cluster selector fits k-means clusters in the learned latent space with k matched to the number of occupied F_{top8} families for that checkpoint. The random selector permutes count-matched labels on the fitting transitions and routes later states by nearest random-label latent center. The reported quantity is the routed/global MSE ratio at $h = 100$, so values below one mean that the selected local maps improve over the same model's global rollout. Because this appendix table reports arithmetic means of per-system seed-IQM finite ratios, rather than the main routing table's system-level mean log effect, isolated catastrophic finite rollouts can dominate all-state cells; the deep-state slice is the cleaner partition audit.

The control table rules out the strongest but false interpretation of the routing result: basin labels alone are not a guaranteed route to a stable closed-loop local predictor in an arbitrary learned latent space. Oracle basin-label maps are very strong for the LISTA-style rows in this audit table, which shows that useful local centered laws exist for those learned representations. The same oracle partition is unstable for the MLP controls, including the dense no-sparsity MLP. Thus the supported conclusion is narrower and more precise: route-local centered maps can help, but the partition, the latent geometry, and the autonomous rollout dynamics must be compatible.

The non-oracle support-family rows show why F_{top8} remains a meaningful mechanism variable. On basin-interior initial states, F_{top8}-local routing gives large improvements for the LISTA-style rows in the table, while the dense no-sparsity MLP support-family route fails catastrophically. This supports the main claim that sparse encoders produce inspectable active-coordinate families that can act as label-free routing variables. The latent-cluster controls show that generic latent geometry can also support useful local maps, especially for sparse MLP controls, so the result should not be read as "only support families can ever route." The random count-matched controls can give modest gains in some rows, but they do not reproduce the strongest deep-state support-family or latent-cluster effects and they fail for the dense MLP. The take-away is therefore comparative rather than absolute: F_{top8} is a label-free, interpretable selector produced by the sparse representation, whereas oracle basins, k-means clusters, and random matched partitions are diagnostic controls rather than deployment-time support objects.

A separate routed-forecasting add-on compared the same post-hoc F_{top8} family-local centered predictor against the same model's best-periodic paper-table forecasts. The fully autonomous routed

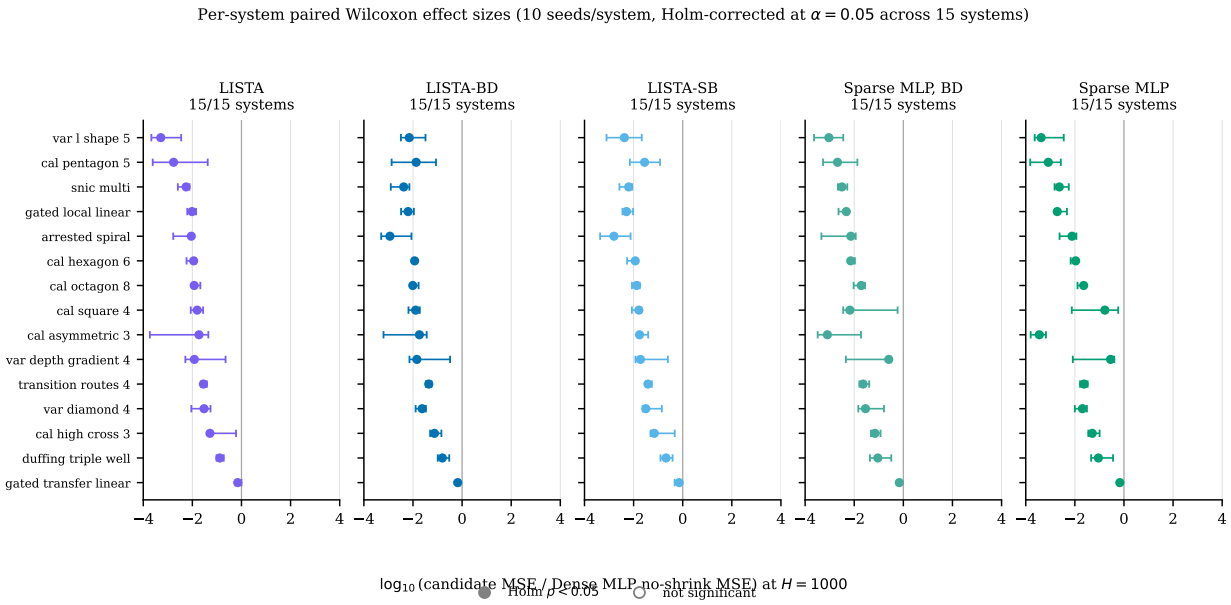


Figure 7: Per-system effect sizes at H=1000 versus the Dense MLP baseline. Each point is one benchmark system; the x-coordinate is the per-system median paired $\log_{10}$-MSE difference (candidate minus baseline) over the common completed seeds, up to 15 per system, with whiskers showing the 95% bootstrap interval. Filled markers clear Holm-corrected α=0.05. Numerical values are in table 11.

version was negative for absolute forecasting superiority: it produced complete merged outputs for the retained controlled multibasin benchmark and Dysts benchmark with 0 failures, but for every $d_z = 256$ LISTA-family model and evaluated horizon, routed rows won 0/15 controlled systems and 0/10 Dysts systems against the same-model best-periodic baseline. A follow-up that periodically decoded and re-encoded the routed rollout every 5, 10, 20, or 30 steps also completed with 0 failures. Periodic refresh reduced many autonomous blow-ups and made route coverage high on the controlled systems, but it did not reverse the forecasting conclusion: every aggregate routed/best-periodic ratio remained above one, with only isolated system wins in individual cells. This negative result is important because the main routing table compares routed rollouts to the same model's autonomous global-K rollout, whereas the paper-table forecasting baseline is stronger because it periodically decodes and re-encodes. We therefore use support-family routing as mechanism evidence that F_{top8} can select useful local predictors relative to the same model's global latent rollout, not as evidence that the post-hoc routed predictor replaces best-periodic forecasting in the aggregate forecasting tables.

H Full per-system multibasin tables

Tables 9–11 render the per-(system, candidate) effect sizes and Holm-corrected Wilcoxon p-values that underlie the K/N counts in table 1 and the forest plot in figure 7. Each cell gives the per-system median paired $\log_{10}$-MSE difference (candidate minus matched-sampling Dense MLP, no-shrink baseline) over the common completed seeds, up to 15 per system, and the corresponding Holm-corrected one-sided paired Wilcoxon p-value. Bold entries indicate Holm-corrected $p < 0.05$ in favor of the candidate.

Per-system audit of the interpretability columns

Why the wrong-support denominator is 11 and not 15. The wrong-support ablation builds, for each system, a dictionary mapping each basin to its canonical support mask, defined as the most-common exact support among the deep-slice states of that basin. The ablated rollout draws a mask from a basin different from the state's own basin and holds that wrong mask fixed, so the test requires at least two distinct deep-slice canonical masks. Four of the 15 retained benchmark systems, the arrested spiral, asymmetric three-well, triple-well Duffing, and L-shaped five-well systems, have multiple basins by design but a global top-quartile deep slice concentrated in a single basin. The wrong-support ablation is undefined on those four systems by construction, and the corresponding K/N denominator in table 1 and tables 14 and 15 is 11 instead of 15. A per-basin top-quartile depth re-evaluation is the appendix robustness analysis for the alternative basin-stratified coverage question, but the global-slice numbers remain the source of the current main table.

Tables 12–15 render the full per-(system, candidate) breakdown of the retained active-count diagnostic and the three directional interpretability columns of table 1. The alternative direction is "candidate < baseline" for $|S_{\text{abs}}|$ and $H(B \mid F_{\text{abs}})$, and "candidate > baseline" for the wrong-support ablation ratios. The main-text family-count column is intentionally not included in these one-sided tests because $|F_{\text{abs}}|$ is a non-directional count.

I Per-seed distributions

Table 1 reports arithmetic means across retained systems after per-system seed-IQM summaries. The corresponding per-(system, seed) distributions are shown below. Figure 8 shows the paper-facing strip plot for the matched boundary-emphasized rows. Figure 9 plots per-(system, seed) histograms of forecasting MSE at every paper-facing horizon. Figure 10 plots per-(system, seed) histograms of $H(B \mid S_{\text{abs}})$, $H(S_{\text{abs}} \mid B)$, and exact-support uniqueness on the deep-state slice. Figure 11 plots the same per-seed distributions for the Dysts $dt{\times}30$ benchmark. The histograms show retained-system and seed-level dispersion around the paper-facing means.

J Support-routing robustness

Table 16 reports route-form diagnostics that are not the main table. The deep slice removes boundary-adjacent states, so it is useful as a robustness check rather than the primary deployment setting. The exact S_{top8} columns should be read as ablations of the support-family router rather than as the headline routing object.

For the gated exact-support diagnostic, let $m_c \in \{0,1\}^{d_z}$ be the eight-coordinate mask of the exact route $c = S_{\text{top8}}(z_t)$, and let $\bar{z}_c$ be the corresponding fitting-set mean. The gated rule masks the centered latent state before applying the learned global transition:

$$\hat{z}_{t+1} = \bar{z}_c + K\big(m_c \odot (z_t - \bar{z}_c)\big). \tag{34}$$

This diagnostic asks whether the active coordinates select useful columns or subspaces of the learned transition. The family-local route in the main text asks a different question: whether the same support family can key a post-hoc local linear map.

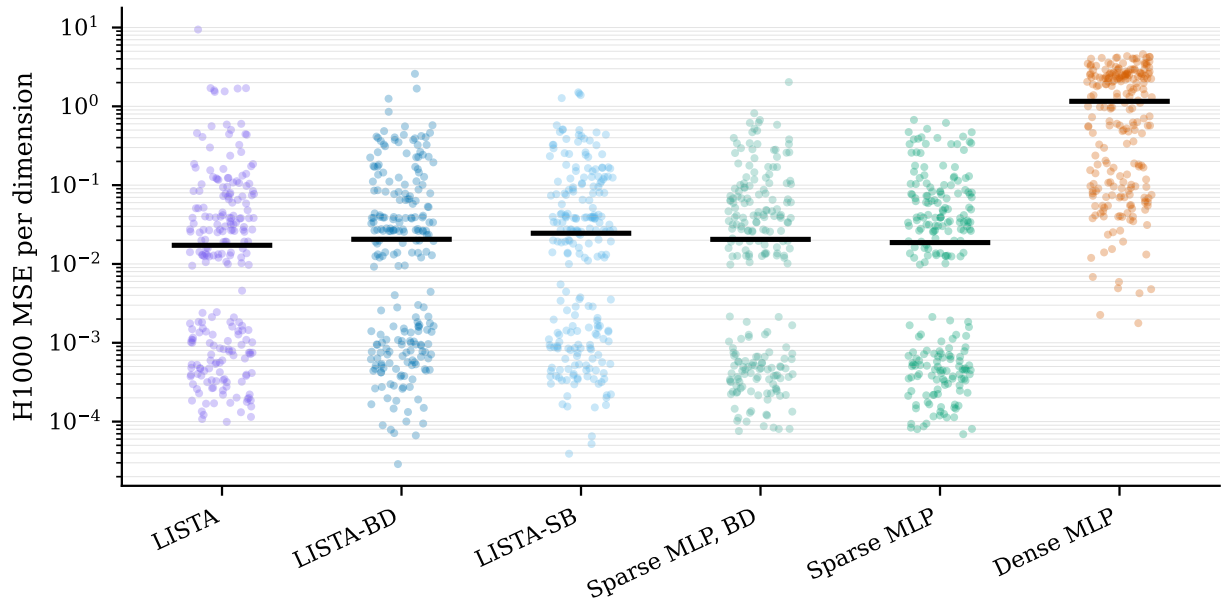


Figure 8: Per-(system, seed) forecasting MSE at $H100$, $H500$, and $H1000$ on the multibasin benchmark. Black bars mark the mean across per-system seed-IQM values. The Dense MLP has a much heavier upper tail at every horizon.

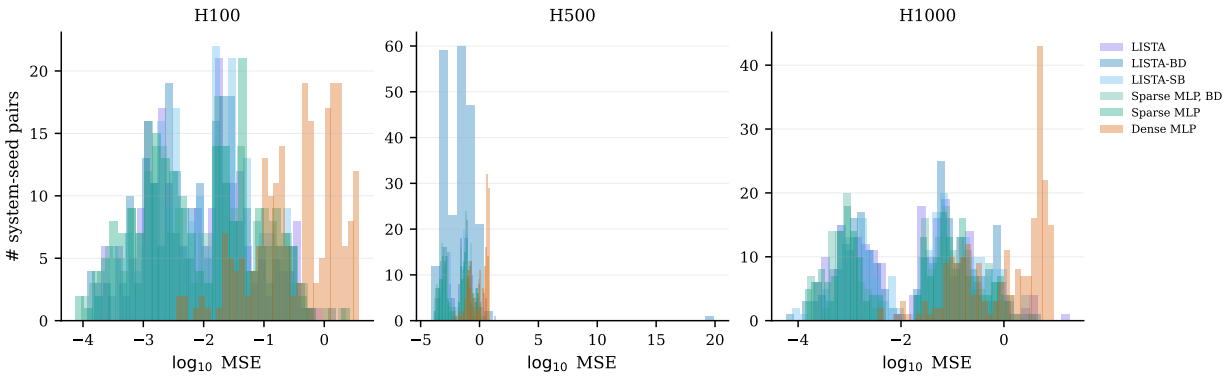


Figure 9: Per-(system, seed) forecasting MSE distributions on the multibasin benchmark. Each row is a horizon; histograms are over the $\log_{10}$ raw MSE.

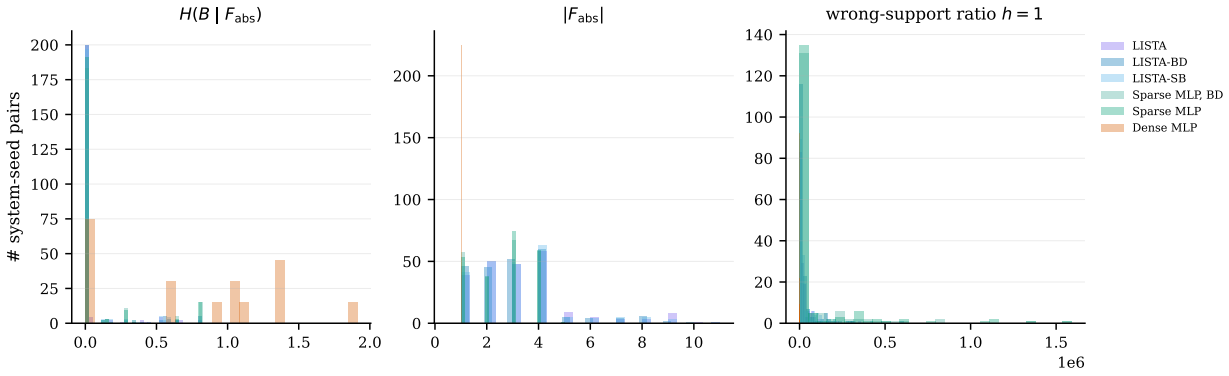


Figure 10: Per-(system, seed) support–basin diagnostic distributions on the deep-state slice. Histograms show $H(B \mid S_{\text{abs}})$, $H(S_{\text{abs}} \mid B)$, and exact-support uniqueness for the rows summarized in table 1.

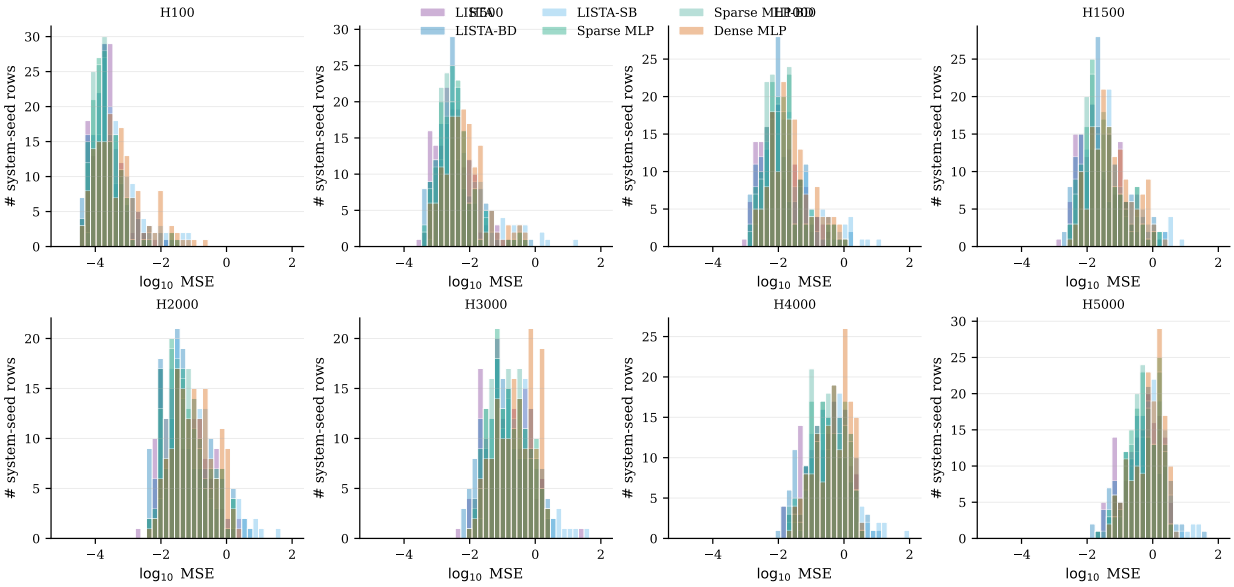


Figure 11: Per-(system, seed) forecasting MSE distributions on the Dysts $dt\times30$ benchmark.

K Dysts forecasting diagnostics

The per-(system, seed) Dysts MSE values are released in the collected `forecasting_rows.csv` artifacts. Table 3 reports arithmetic means across systems after first summarizing seeds within each system by IQM. The $dt\times30$ packet is fully finite in median coverage at every reported horizon, but the per-seed distributions remain heavy-tailed on several systems, so the main table and figures 3b and 11 should be read together: the table gives actual-MSE point estimates and paired system-level significance markers, while the figures show horizon-scale uncertainty and per-seed dispersion. The soft-block LISTA diagnostic improves on Dense MLP in aggregate but does not match the primary sparse forecasting rows and does not survive the main Dysts significance correction. The older tiny-step $H\leq60000$ packet is useful as a stress-test diagnostic because it exposed divergent block-diagonal LISTA rollouts, but it is not pooled with the coarser-step table.

L Additional background

L.1 Koopman operators and finite-dimensional approximations

Consider the stored-step map used in section 2,

$$x_{k+1} = F_{\Delta t}(x_k), \qquad x_k \in \mathcal{X} \subseteq \mathbb{R}^{d_x}. \tag{35}$$

Koopman operator theory studies the evolution of observables rather than the evolution of the state directly. An observable is a measurement function $g : \mathcal{X} \to \mathbb{R}$, and the Koopman operator $\mathcal{K}$ acts by composition:

$$(\mathcal{K}g)(x) = g(F_{\Delta t}(x)). \tag{36}$$

The map $F_{\Delta t}$ may be nonlinear, but $\mathcal{K}$ is linear on the function space of observables. If a finite vector of observables

$$\Phi(x) = \big[g_1(x), \dots, g_{d_z}(x)\big]^\top \tag{37}$$

spans a Koopman-invariant subspace, then there is a matrix $K \in \mathbb{R}^{d_z\times d_z}$ such that

$$\Phi(x_{k+1}) = \Phi(F_{\Delta t}(x_k)) = \mathcal{K}\Phi(x_k) = K\Phi(x_k). \tag{38}$$

In that case the nonlinear dynamics are exactly linear in the observable coordinates $z_k = \Phi(x_k)$. Learning Koopman models can therefore be viewed as searching for observables whose span is invariant enough to support useful finite-dimensional linear prediction.

In practice, the infinite-dimensional operator is approximated from data. Given snapshot pairs $\{(x_k, x_{k+1})\}$ and a chosen feature map Φ, dynamic mode decomposition and extended dynamic mode decomposition (eDMD) estimate a finite transition by least squares (Schmid, 2010; Rowley et al., 2009; Tu et al., 2014; Williams et al., 2015, 2016):

$$K = \operatorname*{arg\,min}_{\widetilde{K}\in\mathbb{R}^{d_z\times d_z}} \sum_k \Big\|\Phi(x_{k+1}) - \widetilde{K}\Phi(x_k)\Big\|_2^2. \tag{39}$$

DMD is the special case $\Phi(x) = x$, so it fits a linear state-space surrogate without a learned nonlinear lift. eDMD extends this by choosing a richer observable dictionary. Koopman autoencoders replace the fixed observable dictionary with a learned encoder and decoder, which is the setting used in this paper.

L.2 Why multibasin systems require local structure

The theoretical obstruction in multibasin systems is not the absence of local linear descriptions. Near appropriate attracting sets, classical linearization and Koopman eigenfunction constructions can produce useful local or basin-restricted coordinates. The obstruction is instead global: as stated in the main text, multi-attractor systems should not generally be expected to admit a single finite-dimensional global linearization with a continuous encoder-decoder pair (Lan and Mezić, 2013; Brunton et al., 2016; Pan and Duraisamy, 2024; Liu et al., 2025). A single continuous, exact, state-reconstructing finite-dimensional Koopman representation is therefore compatible only with restricted global attractor structure.

For the present paper, this motivates a cautious formulation. A learned finite-dimensional Koopman representation on a multibasin system should not be expected to be one exact global linearization that reconstructs every basin simultaneously. It must either approximate across incompatible regions, or it must contain some implicit information about which local description is active. Sparse supports are useful because they make one such piece of information discrete and inspectable: the support can identify a basin, basin part, or local predictive chart while the coefficient values carry within-chart state information.

L.3 Koopman autoencoders and periodic re-encoding

A standard Koopman autoencoder learns an encoder $E_\theta : \mathbb{R}^{d_x} \to \mathbb{R}^{d_z}$, decoder $D_\phi : \mathbb{R}^{d_z} \to \mathbb{R}^{d_x}$, and latent transition K so that

$$z_k = E_\theta(x_k), \qquad z_{k+1} \approx K z_k, \qquad x_k \approx D_\phi(z_k). \tag{40}$$

A schematic one-step objective has the form

$$\min_{E_\theta, D_\phi, K} \sum_k \|x_k - D_\phi(E_\theta(x_k))\|_2^2 + \lambda_{\text{lin}} \|E_\theta(x_{k+1}) - K E_\theta(x_k)\|_2^2 . \tag{41}$$

This objective is useful as a reference point, but it is not sufficient for the present study. It does not directly train decoded multi-step rollouts, does not impose a sparse support object, and does not test whether the support itself selects a useful local predictive rule. The training objective in section 3.3 therefore keeps the autoencoding and latent-linearity terms but adds decoded rollout prediction over windows, sparsity on the rolled latent states, and optional transition-structure penalties.

Periodic re-encoding interrupts a latent rollout after a fixed number of stored observation steps. For period m,

$$\tilde{z}_{k+m} = K^m z_k, \qquad \tilde{x}_{k+m} = D_\phi(\tilde{z}_{k+m}), \qquad z_{k+m} = E_\theta(\tilde{x}_{k+m}). \tag{42}$$

This is the same mechanism defined in equation (16). It uses only the model's own predicted state and is therefore not a teacher-forced correction. The point is to refresh the latent embedding and, in sparse models, to refresh the support as the predicted state moves through regions where a different local linear description may be more appropriate (Fathi et al., 2024).

L.4 Sparse coding and LISTA

Classical sparse coding seeks a code z that reconstructs a signal from a small set of active dictionary atoms:

$$z^\star(x) = \arg\min_{z \in \mathbb{R}^{d_z}} \frac{1}{2} \|x - Dz\|_2^2 + \lambda_{\text{sc}} \|z\|_1 , \qquad \lambda_{\text{sc}} > 0, \tag{43}$$

where D is a dictionary and λ_{sc} controls sparsity (Olshausen and Field, 1996; Chen et al., 1998; Donoho and Elad, 2003). For a fixed dictionary, and under the usual uniqueness and non-boundary conditions for the Lasso solution, the active set and signs are locally constant over regions of the input space, while the coefficient values vary continuously within a fixed active set. The decoder reconstruction then lies in the span of the selected dictionary columns, giving a union-of-subspaces geometry. Each exact support can index a local chart, and the greedy Jaccard procedure in section 3.4 groups nearby Boolean support vectors into support families when threshold noise or boundary effects fragment the exact masks.

LISTA replaces an iterative sparse-coding solver with a learned finite-depth shrinkage network (Gregor and LeCun, 2010). In the notation of equation (7), the encoder forms a pre-code from the input and repeatedly applies learned affine refinement followed by soft thresholding. In this paper, the encoder is trained jointly with the Koopman rollout objective rather than solving the sparse-coding problem at evaluation time. The thresholding operation is the important ingredient: it turns the latent code into both continuous coefficients and an explicit support. The experiments then ask whether F_{abs} groups absolute-threshold supports S_{abs} in a way that aligns with withheld benchmark basin labels, whether S_{abs} is functionally used by the predictor, whether F_{top8} can route family-local predictors, and whether refreshed S_{top8} supports track controlled basin transfer.

References

Rishabh Agarwal, Max Schwarzer, Pablo Samuel Castro, Aaron C Courville, and Marc Bellemare. Deep reinforcement learning at the edge of the statistical precipice. *Advances in Neural Information Processing Systems*, 34, 2021.

Omri Azencot, N. Benjamin Erichson, Vanessa Lin, and Michael Mahoney. Forecasting Sequential Data Using Consistent Koopman Autoencoders. In *Proceedings of the 37th International Conference on Machine Learning*, pages 475–485. PMLR, November 2020.

Amir Beck and Marc Teboulle. A fast iterative shrinkage-thresholding algorithm for linear inverse problems. *SIAM Journal on Imaging Sciences*, 2(1):183–202, 2009. doi: 10.1137/080716542.

Steven L. Brunton, Bingni W. Brunton, Joshua L. Proctor, and J. Nathan Kutz. Koopman Invariant Subspaces and Finite Linear Representations of Nonlinear Dynamical Systems for Control. *PLOS ONE*, 11(2), February 2016. ISSN 1932-6203. doi: 10.1371/journal.pone.0150171. URL https://journals.plos.org/plosone/article?id=10.1371/journal.pone.0150171.

Steven L. Brunton, Marko Budišić, Eurika Kaiser, and J. Nathan Kutz. Modern Koopman Theory for Dynamical Systems. *SIAM Review*, 64(2):229–340, May 2022. ISSN 0036-1445. doi: 10.1137/21M1401243. URL https://doi.org/10.1137/21M1401243.

Scott Shaobing Chen, David L. Donoho, and Michael A. Saunders. Atomic Decomposition by Basis Pursuit. *SIAM Journal on Scientific Computing*, 20(1):33–61, 1998. doi: 10.1137/S1064827596304010. URL https://doi.org/10.1137/S1064827596304010.

David L. Donoho and Michael Elad. Optimally sparse representation in general (nonorthogonal) dictionaries via ℓ_1 minimization. *Proceedings of the National Academy of Sciences*, 100(5): 2197–2202, March 2003. doi: 10.1073/pnas.0437847100. URL https://doi.org/10.1073/pnas.0437847100.

Mahan Fathi, Clement Gehring, Jonathan Pilault, David Kanaa, Pierre-Luc Bacon, and Ross Goroshin. Course correcting koopman representations. In *The Twelfth International Conference on Learning Representations*, 2024. URL https://openreview.net/forum?id=A18gWgc5mi.

William Gilpin. Chaos as an interpretable benchmark for forecasting and data-driven modelling. In *Advances in Neural Information Processing Systems (NeurIPS) 2021*, 2021. URL http://arxiv.org/abs/2110.05266.

William Gilpin. Model scale versus domain knowledge in statistical forecasting of chaotic systems. *Physical Review Research*, 5(4):043252, December 2023. doi: 10.1103/PhysRevResearch.5.043252. URL https://link.aps.org/doi/10.1103/PhysRevResearch.5.043252.

Karol Gregor and Yann LeCun. Learning Fast Approximations of Sparse Coding. In *Proceedings of the 27th International Conference on Machine Learning*, ICML'10, pages 399–406, Haifa, Israel, 2010. Omnipress. ISBN 978-1-60558-907-7. doi: 10.5555/3104322.3104374. URL https://dl.acm.org/doi/abs/10.5555/3104322.3104374.

D. M. Grobman. Homeomorphism of systems of differential equations. *Doklady Akademii Nauk SSSR*, 128:880–881, 1959.

Lars Grüne and Jürgen Pannek. *Nonlinear Model Predictive Control*. Communications and Control Engineering. Springer International Publishing, Cham, 2017. ISBN 978-3-319-46023-9 978-3-319-46024-6. doi: 10.1007/978-3-319-46024-6. URL http://link.springer.com/10.1007/978-3-319-46024-6.

John A. Hartigan. *Clustering Algorithms*. John Wiley & Sons, New York, 1975. ISBN 0-471-35645-X.

Philip Hartman. A lemma in the theory of structural stability of differential equations. *Proceedings of the American Mathematical Society*, 11(4):610–620, 1960. doi: 10.1090/S0002-9939-1960-0121542-7.

Paul Jaccard. Étude comparative de la distribution florale dans une portion des alpes et du jura. *Bulletin de la Société Vaudoise des Sciences Naturelles*, 37(142):547–579, 1901. doi: 10.5169/seals-266450.

R.E. Kalman. On the general theory of control systems. *1st International IFAC Congress on Automatic and Remote Control, Moscow, USSR, 1960*, 1(1):491–502, August 1960. ISSN 1474-6670. doi: 10.1016/S1474-6670(17)70094-8. URL https://www.sciencedirect.com/science/article/pii/S1474667017700948.

B. O. Koopman. Hamiltonian Systems and Transformation in Hilbert Space. *Proceedings of the National Academy of Sciences*, 17(5):315–318, May 1931. doi: 10.1073/pnas.17.5.315. URL https://www.pnas.org/doi/10.1073/pnas.17.5.315.

B. O. Koopman and J. v. Neumann. Dynamical Systems of Continuous Spectra. *Proceedings of the National Academy of Sciences*, 18(3):255–263, March 1932. doi: 10.1073/pnas.18.3.255. URL https://www.pnas.org/doi/abs/10.1073/pnas.18.3.255.

Milan Korda and Igor Mezić. Linear predictors for nonlinear dynamical systems: Koopman operator meets model predictive control. *Automatica*, 93:149–160, July 2018. ISSN 00051098. doi: 10.1016/j.automatica.2018.03.046. URL http://arxiv.org/abs/1611.03537.

Matthew D. Kvalheim and Shai Revzen. Existence and uniqueness of global Koopman eigenfunctions for stable fixed points and periodic orbits. *Physica D: Nonlinear Phenomena*, 425:132959, November 2021. ISSN 0167-2789. doi: 10.1016/j.physd.2021.132959. URL https://www.sciencedirect.com/science/article/pii/S0167278921001160.

Yueheng Lan and Igor Mezić. Linearization in the large of nonlinear systems and Koopman operator spectrum. *Physica D: Nonlinear Phenomena*, 242(1):42–53, January 2013. ISSN 0167-2789. doi: 10.1016/j.physd.2012.08.017.

Zexiang Liu, Necmiye Ozay, and Eduardo D. Sontag. Properties of immersions for systems with multiple limit sets with implications to learning Koopman embeddings. *Automatica*, 176: 112226, June 2025. ISSN 0005-1098. doi: 10.1016/j.automatica.2025.112226. URL https://www.sciencedirect.com/science/article/pii/S0005109825001189.

Bethany Lusch, J. Nathan Kutz, and Steven L. Brunton. Deep learning for universal linear embeddings of nonlinear dynamics. *Nature Communications*, 9(1):4950, November 2018. ISSN 2041-1723. doi: 10.1038/s41467-018-07210-0.

Giorgos Mamakoukas, Maria Castano, Xiaobo Tan, and Todd Murphey. Local Koopman Operators for Data-Driven Control of Robotic Systems. In *Robotics: Science and Systems XV*. Robotics: Science and Systems Foundation, June 2019. ISBN 978-0-9923747-5-4. doi: 10.15607/RSS.2019.XV.054. URL http://www.roboticsproceedings.org/rss15/p54.pdf.

D. Q. Mayne, J. B. Rawlings, C. V. Rao, and P. O. M. Scokaert. Constrained model predictive control: Stability and optimality. *Automatica*, 36(6):789–814, June 2000. ISSN 0005-1098. doi: 10.1016/S0005-1098(99)00214-9. URL https://www.sciencedirect.com/science/article/pii/S0005109899002149.

Igor Mezić. Spectral properties of dynamical systems, model reduction and decompositions. *Nonlinear Dynamics*, 41(1–3):309–325, 2005. doi: 10.1007/s11071-005-2824-x.

Bruno A. Olshausen and David J. Field. Emergence of simple-cell receptive field properties by learning a sparse code for natural images. *Nature*, 381(6583):607–609, June 1996. ISSN 1476-4687. doi: 10.1038/381607a0. URL https://doi.org/10.1038/381607a0.

Samuel E. Otto and Clarence W. Rowley. Linearly recurrent autoencoder networks for learning dynamics. *SIAM Journal on Applied Dynamical Systems*, 18(1):558–593, 2019. doi: 10.1137/18M1177846.

Shaowu Pan and Karthik Duraisamy. On the lifting and reconstruction of nonlinear systems with multiple invariant sets. *Nonlinear Dynamics*, 112(12):10157–10165, June 2024. ISSN 1573-269X. doi: 10.1007/s11071-024-09581-0. URL https://doi.org/10.1007/s11071-024-09581-0.

Clarence W. Rowley, Igor Mezić, Shervin Bagheri, Philipp Schlatter, and Dan S. Henningson. Spectral analysis of nonlinear flows. *Journal of Fluid Mechanics*, 641:115–127, 2009. doi: 10.1017/S0022112009992059.

Peter J. Schmid. Dynamic mode decomposition of numerical and experimental data. *Journal of Fluid Mechanics*, 656:5–28, 2010. ISSN 0022-1120. doi: 10.1017/S0022112010001217.

Haojie Shi and Max Q.-H. Meng. Deep Koopman Operator With Control for Nonlinear Systems. *IEEE Robotics and Automation Letters*, 7(3):7700–7707, July 2022. ISSN 2377-3766. doi: 10.1109/LRA.2022.3184036.

Naoya Takeishi, Yoshinobu Kawahara, and Takehisa Yairi. Learning koopman invariant subspaces for dynamic mode decomposition. In *Advances in Neural Information Processing Systems 30*, volume 30, 2017. URL https://papers.nips.cc/paper/6713-learning-koopman-invariant-subspaces-for-dynamic-mode-decomposition.

Jonathan H. Tu, Clarence W. Rowley, Dirk M. Luchtenburg, Steven L. Brunton, and J. Nathan Kutz. On dynamic mode decomposition: Theory and applications. *Journal of Computational Dynamics*, 1(2):391–421, December 2014. ISSN 2158-2491. doi: 10.3934/jcd.2014.1.391. URL https://www.aimsciences.org/en/article/doi/10.3934/jcd.2014.1.391.

Matthew O. Williams, Ioannis G. Kevrekidis, and Clarence W. Rowley. A Data–Driven Approximation of the Koopman Operator: Extending Dynamic Mode Decomposition. *Journal of Nonlinear Science*, 25(6):1307–1346, December 2015. ISSN 1432-1467. doi: 10.1007/s00332-015-9258-5. URL https://doi.org/10.1007/s00332-015-9258-5.

Matthew O. Williams, Clarence W. Rowley, and Ioannis G. Kevrekidis. A kernel-based method for data-driven koopman spectral analysis. *Journal of Computational Dynamics*, 2(2):247–265, May 2016. ISSN 2158-2491. doi: 10.3934/jcd.2015005. URL https://www.aimsciences.org/article/doi/10.3934/jcd.2015005.

Table 8: Partition-control local forecasting for the top-8 support-family comparison. Entries are routed/global MSE ratios, summarized by seed-IQM within each system and arithmetic mean across systems; lower is better. Bracketed cells give within-system Wilcoxon/Holm counts when present. The LISTA-SB row uses the matched p256 (d_z=256) control artifact.

		$H100$	
Model	Selector	All	Deep
LISTA-SB	Oracle basin labels	0.0176	0.00679
	Support family	0.168	0.0374
	Latent clusters	0.179	0.0471
	Random matched	0.857	0.869
LISTA-BD	Oracle basin labels	0.0330 [15/15]	0.0306 [14/15]
	Support family	2.18 [11/15]	0.135 [12/15]
	Latent clusters	2.83 [11/15]	0.368 [12/15]
	Random matched	0.861 [13/15]	0.887 [12/15]
Sparse MLP, BD	Oracle basin labels	2.57×10^{12} [8/15]	2.60×10^{12} [7/15]
	Support family	1.20 [8/15]	0.779 [8/15]
	Latent clusters	0.241 [13/15]	0.230 [13/15]
	Random matched	0.929 [14/15]	0.961 [8/15]
Sparse MLP	Oracle basin labels	6.00×10^{9} [11/15]	1.19×10^{10} [10/15]
	Support family	0.639 [6/15]	0.443 [5/15]
	Latent clusters	0.279 [14/15]	0.221 [13/15]
	Random matched	0.925 [12/15]	0.956 [11/15]
Dense MLP	Oracle basin labels	1.75×10^{33} [4/15]	1.36×10^{33} [4/15]
	Support family	3.32×10^{3} [0/15]	2.61×10^{3} [0/15]
	Latent clusters	5.73×10^{9} [0/15]	9.57×10^{8} [2/15]
	Random matched	179 [0/15]	322 [0/15]

Table 9: Per-system effect sizes and Holm-corrected Wilcoxon p-values at $H = 100$ on the 15-system multibasin benchmark, against the matched-sampling Dense MLP, no-shrink baseline. Δ is the per-system median paired $\log_{10}$-MSE difference (candidate − baseline); p_{H} is the one-sided paired Wilcoxon signed-rank p-value, Holm-corrected across retained systems within each candidate column. Bold Δ: Holm-corrected $p < 0.05$ in favor of the candidate.

System	LISTA		LISTA-BD		LISTA-SB		Sparse MLP, BD		Sparse MLP	
	Δ	p_{H}	Δ	p_{H}	Δ	p_{H}	Δ	p_{H}	Δ	p_{H}
var_l_shape_5	**−2.23**	$<10^{-3}$	**−1.94**	$<10^{-3}$	**−1.90**	$<10^{-3}$	**−2.05**	$<10^{-3}$	**−2.23**	$<10^{-3}$
cal_pentagon_5	**−1.86**	$<10^{-3}$	**−1.58**	$<10^{-3}$	**−1.20**	$<10^{-3}$	**−1.89**	$<10^{-3}$	**−2.16**	$<10^{-3}$
snic_multi	**−2.01**	$<10^{-3}$	**−2.03**	$<10^{-3}$	**−1.91**	$<10^{-3}$	**−2.09**	$<10^{-3}$	**−2.13**	$<10^{-3}$
gated_local_linear	**−2.16**	$<10^{-3}$	**−2.19**	$<10^{-3}$	**−2.20**	$<10^{-3}$	**−2.35**	$<10^{-3}$	**−2.52**	$<10^{-3}$
arrested_spiral	**−1.85**	$<10^{-3}$	**−2.09**	$<10^{-3}$	**−2.03**	$<10^{-3}$	**−1.93**	$<10^{-3}$	**−1.90**	$<10^{-3}$
cal_hexagon_6	**−1.76**	$<10^{-3}$	**−1.73**	$<10^{-3}$	**−1.85**	$<10^{-3}$	**−1.96**	$<10^{-3}$	**−1.83**	$<10^{-3}$
cal_octagon_8	**−1.85**	$<10^{-3}$	**−1.90**	$<10^{-3}$	**−1.80**	$<10^{-3}$	**−1.68**	$<10^{-3}$	**−1.63**	$<10^{-3}$
cal_square_4	**−1.44**	$<10^{-3}$	**−1.62**	$<10^{-3}$	**−1.58**	$<10^{-3}$	**−1.39**	$<10^{-3}$	**−1.02**	$<10^{-3}$
cal_asymmetric_3	**−2.00**	$<10^{-3}$	**−1.92**	$<10^{-3}$	**−1.75**	$<10^{-3}$	**−2.00**	$<10^{-3}$	**−2.44**	$<10^{-3}$
var_depth_gradient_4	**−1.39**	$<10^{-3}$	**−1.33**	$<10^{-3}$	**−1.37**	$<10^{-3}$	**−1.05**	$<10^{-3}$	**−0.82**	$<10^{-3}$
transition_routes_4	**−2.21**	$<10^{-3}$	**−1.77**	$<10^{-3}$	**−1.90**	$<10^{-3}$	**−2.36**	$<10^{-3}$	**−2.47**	$<10^{-3}$
var_diamond_4	**−2.00**	$<10^{-3}$	**−1.80**	$<10^{-3}$	**−1.84**	$<10^{-3}$	**−1.46**	$<10^{-3}$	**−1.67**	$<10^{-3}$
cal_high_cross_3	**−1.05**	$<10^{-3}$	**−1.12**	$<10^{-3}$	**−1.19**	$<10^{-3}$	**−1.10**	$<10^{-3}$	**−1.17**	$<10^{-3}$
duffing_triple_well	**−0.54**	$<10^{-3}$	**−0.51**	0.008	**−0.35**	0.007	**−0.81**	0.002	**−0.78**	$<10^{-3}$
gated_transfer_linear	**−0.27**	0.001	**−0.21**	0.008	**−0.31**	0.007	**−0.36**	$<10^{-3}$	**−0.27**	0.004
$K/15$ *Holm* $p < 0.05$	K=15 / N=15		K=15 / N=15		K=15 / N=15		K=15 / N=15		K=15 / N=15	

Table 10: Per-system effect sizes and Holm-corrected Wilcoxon p-values at $H = 500$ on the 15-system multibasin benchmark. Format identical to table 9.

System	LISTA		LISTA-BD		LISTA-SB		Sparse MLP, BD		Sparse MLP	
	Δ	p_{H}	Δ	p_{H}	Δ	p_{H}	Δ	p_{H}	Δ	p_{H}
var_l_shape_5	**−2.98**	$<10^{-3}$	**−2.04**	$<10^{-3}$	**−2.03**	$<10^{-3}$	**−2.73**	$<10^{-3}$	**−2.96**	$<10^{-3}$
cal_pentagon_5	**−2.48**	$<10^{-3}$	**−2.10**	$<10^{-3}$	**−1.61**	$<10^{-3}$	**−2.42**	$<10^{-3}$	**−2.77**	$<10^{-3}$
snic_multi	**−2.30**	$<10^{-3}$	**−2.45**	$<10^{-3}$	**−2.31**	$<10^{-3}$	**−2.44**	$<10^{-3}$	**−2.49**	$<10^{-3}$
gated_local_linear	**−2.01**	$<10^{-3}$	**−2.24**	$<10^{-3}$	**−2.27**	$<10^{-3}$	**−2.29**	$<10^{-3}$	**−2.52**	$<10^{-3}$
arrested_spiral	**−2.26**	$<10^{-3}$	**−2.57**	$<10^{-3}$	**−2.57**	$<10^{-3}$	**−2.40**	$<10^{-3}$	**−2.33**	$<10^{-3}$
cal_hexagon_6	**−2.01**	$<10^{-3}$	**−2.01**	$<10^{-3}$	**−2.18**	$<10^{-3}$	**−2.32**	$<10^{-3}$	**−2.16**	$<10^{-3}$
cal_octagon_8	**−1.94**	$<10^{-3}$	**−2.09**	$<10^{-3}$	**−1.91**	$<10^{-3}$	**−1.81**	$<10^{-3}$	**−1.74**	$<10^{-3}$
cal_square_4	**−1.60**	$<10^{-3}$	**−1.89**	$<10^{-3}$	**−1.81**	$<10^{-3}$	**−1.67**	$<10^{-3}$	**−1.17**	$<10^{-3}$
cal_asymmetric_3	**−2.27**	$<10^{-3}$	**−2.12**	$<10^{-3}$	**−1.94**	$<10^{-3}$	**−2.53**	$<10^{-3}$	**−3.15**	$<10^{-3}$
var_depth_gradient_4	**−1.63**	$<10^{-3}$	**−1.45**	$<10^{-3}$	**−1.47**	$<10^{-3}$	**−1.21**	$<10^{-3}$	**−0.92**	$<10^{-3}$
transition_routes_4	**−1.73**	$<10^{-3}$	**−1.52**	$<10^{-3}$	**−1.49**	$<10^{-3}$	**−1.68**	$<10^{-3}$	**−1.86**	$<10^{-3}$
var_diamond_4	**−1.77**	$<10^{-3}$	**−1.58**	$<10^{-3}$	**−1.40**	$<10^{-3}$	**−1.41**	$<10^{-3}$	**−1.75**	$<10^{-3}$
cal_high_cross_3	**−0.93**	$<10^{-3}$	**−0.97**	$<10^{-3}$	**−1.03**	$<10^{-3}$	**−1.17**	$<10^{-3}$	**−1.26**	$<10^{-3}$
duffing_triple_well	**−0.83**	$<10^{-3}$	**−0.81**	$<10^{-3}$	**−0.65**	$<10^{-3}$	**−1.07**	$<10^{-3}$	**−1.05**	$<10^{-3}$
gated_transfer_linear	**−0.08**	0.013	**+1.19**	0.008	**−0.21**	$<10^{-3}$	**−0.18**	$<10^{-3}$	**−0.18**	$<10^{-3}$
$K/15$ *Holm* $p < 0.05$	K=15 / N=15		K=15 / N=15		K=15 / N=15		K=15 / N=15		K=15 / N=15	

Table 11: Per-system effect sizes and Holm-corrected Wilcoxon p-values at $H = 1000$ on the 15-system multibasin benchmark. This is the numerical version of the forest plot in figure 7. Format identical to table 9.

System	LISTA		LISTA-BD		LISTA-SB		Sparse MLP, BD		Sparse MLP	
	Δ	p_{H}	Δ	p_{H}	Δ	p_{H}	Δ	p_{H}	Δ	p_{H}
var_l_shape_5	**−3.29**	$<10^{-3}$	**−2.15**	$<10^{-3}$	**−2.38**	$<10^{-3}$	**−3.03**	$<10^{-3}$	**−3.37**	$<10^{-3}$
cal_pentagon_5	**−2.76**	$<10^{-3}$	**−1.87**	$<10^{-3}$	**−1.55**	$<10^{-3}$	**−2.68**	$<10^{-3}$	**−3.08**	$<10^{-3}$
snic_multi	**−2.26**	$<10^{-3}$	**−2.38**	$<10^{-3}$	**−2.20**	$<10^{-3}$	**−2.50**	$<10^{-3}$	**−2.63**	$<10^{-3}$
gated_local_linear	**−2.01**	$<10^{-3}$	**−2.20**	$<10^{-3}$	**−2.29**	$<10^{-3}$	**−2.32**	$<10^{-3}$	**−2.72**	$<10^{-3}$
arrested_spiral	**−2.04**	$<10^{-3}$	**−2.94**	$<10^{-3}$	**−2.80**	$<10^{-3}$	**−2.14**	$<10^{-3}$	**−2.12**	$<10^{-3}$
cal_hexagon_6	**−1.95**	$<10^{-3}$	**−1.93**	$<10^{-3}$	**−1.93**	$<10^{-3}$	**−2.14**	$<10^{-3}$	**−1.97**	$<10^{-3}$
cal_octagon_8	**−1.92**	$<10^{-3}$	**−2.01**	$<10^{-3}$	**−1.87**	$<10^{-3}$	**−1.72**	$<10^{-3}$	**−1.64**	$<10^{-3}$
cal_square_4	**−1.80**	$<10^{-3}$	**−1.89**	$<10^{-3}$	**−1.78**	$<10^{-3}$	**−2.18**	$<10^{-3}$	**−0.78**	$<10^{-3}$
cal_asymmetric_3	**−1.73**	$<10^{-3}$	**−1.74**	$<10^{-3}$	**−1.76**	$<10^{-3}$	**−3.09**	$<10^{-3}$	**−3.45**	$<10^{-3}$
var_depth_gradient_4	**−1.92**	$<10^{-3}$	**−1.84**	$<10^{-3}$	**−1.72**	$<10^{-3}$	**−0.59**	$<10^{-3}$	**−0.54**	$<10^{-3}$
transition_routes_4	**−1.55**	$<10^{-3}$	**−1.36**	$<10^{-3}$	**−1.42**	$<10^{-3}$	**−1.64**	$<10^{-3}$	**−1.63**	$<10^{-3}$
var_diamond_4	**−1.52**	$<10^{-3}$	**−1.63**	$<10^{-3}$	**−1.50**	$<10^{-3}$	**−1.54**	$<10^{-3}$	**−1.69**	$<10^{-3}$
cal_high_cross_3	**−1.28**	$<10^{-3}$	**−1.13**	$<10^{-3}$	**−1.16**	$<10^{-3}$	**−1.16**	$<10^{-3}$	**−1.29**	$<10^{-3}$
duffing_triple_well	**−0.88**	$<10^{-3}$	**−0.80**	$<10^{-3}$	**−0.68**	$<10^{-3}$	**−1.04**	$<10^{-3}$	**−1.04**	$<10^{-3}$
gated_transfer_linear	**−0.15**	0.013	**−0.18**	$<10^{-3}$	**−0.14**	$<10^{-3}$	**−0.16**	$<10^{-3}$	**−0.17**	$<10^{-3}$
$K/15$ *Holm* $p < 0.05$	K=15 / N=15		K=15 / N=15		K=15 / N=15		K=15 / N=15		K=15 / N=15	

Table 12: Per-system effect sizes and Holm-corrected Wilcoxon p-values for $|S_{\mathrm{abs}}|$, the mean active-coordinate count under the absolute-threshold support on the deep slice. Alternative: candidate $|S_{\mathrm{abs}}| <$ baseline $|S_{\mathrm{abs}}|$.

System	LISTA		LISTA-BD		LISTA-SB		Sparse MLP, BD		Sparse MLP	
	Δ	p_{H}	Δ	p_{H}	Δ	p_{H}	Δ	p_{H}	Δ	p_{H}
snic_multi	**−175.52**	$<10^{-3}$	**−174.21**	$<10^{-3}$	**−171.07**	$<10^{-3}$	**−170.02**	$<10^{-3}$	**−169.93**	$<10^{-3}$
gated_local_linear	**−168.98**	$<10^{-3}$	**−167.67**	$<10^{-3}$	**−167.29**	$<10^{-3}$	**−159.44**	$<10^{-3}$	**−159.44**	$<10^{-3}$
duffing_triple_well	**−163.57**	$<10^{-3}$	**−163.31**	$<10^{-3}$	**−163.67**	$<10^{-3}$	**−148.02**	$<10^{-3}$	**−148.80**	$<10^{-3}$
transition_routes_4	**−167.29**	$<10^{-3}$	**−162.38**	$<10^{-3}$	**−163.99**	$<10^{-3}$	**−152.85**	$<10^{-3}$	**−152.73**	$<10^{-3}$
cal_asymmetric_3	**−161.77**	$<10^{-3}$	**−158.00**	$<10^{-3}$	**−156.12**	$<10^{-3}$	**−163.00**	$<10^{-3}$	**−162.97**	$<10^{-3}$
cal_pentagon_5	**−162.15**	$<10^{-3}$	**−156.55**	$<10^{-3}$	**−157.12**	$<10^{-3}$	**−141.89**	$<10^{-3}$	**−142.43**	$<10^{-3}$
gated_transfer_linear	**−155.81**	$<10^{-3}$	**−152.76**	$<10^{-3}$	**−155.40**	$<10^{-3}$	**−158.88**	$<10^{-3}$	**−158.43**	$<10^{-3}$
cal_hexagon_6	**−159.19**	$<10^{-3}$	**−155.01**	$<10^{-3}$	**−156.11**	$<10^{-3}$	**−142.86**	$<10^{-3}$	**−142.86**	$<10^{-3}$
arrested_spiral	**−156.00**	$<10^{-3}$	**−151.91**	$<10^{-3}$	**−153.06**	$<10^{-3}$	**−154.19**	$<10^{-3}$	**−154.19**	$<10^{-3}$
cal_square_4	**−153.23**	$<10^{-3}$	**−152.11**	$<10^{-3}$	**−153.07**	$<10^{-3}$	**−140.53**	$<10^{-3}$	**−140.78**	$<10^{-3}$
var_l_shape_5	**−157.68**	$<10^{-3}$	**−151.90**	$<10^{-3}$	**−155.03**	$<10^{-3}$	**−142.74**	$<10^{-3}$	**−142.74**	$<10^{-3}$
var_diamond_4	**−160.07**	$<10^{-3}$	**−144.83**	$<10^{-3}$	**−157.41**	$<10^{-3}$	**−149.83**	$<10^{-3}$	**−149.50**	$<10^{-3}$
var_depth_gradient_4	**−154.59**	$<10^{-3}$	**−149.53**	$<10^{-3}$	**−152.15**	$<10^{-3}$	**−140.31**	$<10^{-3}$	**−140.15**	$<10^{-3}$
cal_octagon_8	**−152.81**	$<10^{-3}$	**−147.55**	$<10^{-3}$	**−152.64**	$<10^{-3}$	**−143.20**	$<10^{-3}$	**−143.34**	$<10^{-3}$
cal_high_cross_3	**−148.93**	$<10^{-3}$	**−143.99**	$<10^{-3}$	**−146.03**	$<10^{-3}$	**−127.04**	$<10^{-3}$	**−133.00**	$<10^{-3}$
$K/15$ *Holm* $p < 0.05$	K=15 / N=15		K=15 / N=15		K=15 / N=15		K=15 / N=15		K=15 / N=15	

Table 13: Per-system effect sizes and Holm-corrected Wilcoxon p-values for $H(B \mid F_{\text{abs}})$ on the deep slice. Alternative: candidate $H(B \mid F_{\text{abs}}) <$ baseline.

System	LISTA		LISTA-BD		LISTA-SB		Sparse MLP, BD		Sparse MLP	
	Δ	p_{H}	Δ	p_{H}	Δ	p_{H}	Δ	p_{H}	Δ	p_{H}
transition_routes_4	**−1.38**	$<10^{-3}$	**−1.38**	$<10^{-3}$	**−1.38**	$<10^{-3}$	**−1.38**	$<10^{-3}$	**−1.38**	$<10^{-3}$
cal_square_4	**−1.37**	$<10^{-3}$	**−1.37**	$<10^{-3}$	**−1.37**	$<10^{-3}$	**−1.37**	$<10^{-3}$	**−1.37**	$<10^{-3}$
var_diamond_4	**−1.37**	$<10^{-3}$	**−1.37**	$<10^{-3}$	**−1.37**	$<10^{-3}$	**−1.37**	$<10^{-3}$	**−1.37**	$<10^{-3}$
cal_octagon_8	**−1.37**	$<10^{-3}$	**−1.33**	$<10^{-3}$	**−1.36**	$<10^{-3}$	**−1.09**	0.001	**−1.09**	$<10^{-3}$
snic_multi	**−1.09**	$<10^{-3}$	**−1.09**	$<10^{-3}$	**−1.09**	$<10^{-3}$	**−1.09**	$<10^{-3}$	**−1.09**	$<10^{-3}$
gated_local_linear	**−1.07**	$<10^{-3}$	**−1.07**	$<10^{-3}$	**−1.07**	$<10^{-3}$	**−1.07**	$<10^{-3}$	**−1.07**	$<10^{-3}$
gated_transfer_linear	**−1.06**	$<10^{-3}$	**−1.06**	$<10^{-3}$	**−1.06**	$<10^{-3}$	**−1.06**	$<10^{-3}$	**−1.06**	$<10^{-3}$
cal_hexagon_6	**−0.92**	$<10^{-3}$	**−0.92**	$<10^{-3}$	**−0.82**	$<10^{-3}$	**−0.66**	0.001	**−0.66**	$<10^{-3}$
cal_pentagon_5	**−0.64**	$<10^{-3}$	**−0.64**	$<10^{-3}$	**−0.64**	$<10^{-3}$	**−0.64**	0.002	**−0.64**	$<10^{-3}$
var_depth_gradient_4	**−0.62**	$<10^{-3}$	**−0.62**	$<10^{-3}$	**−0.62**	$<10^{-3}$	**−0.62**	$<10^{-3}$	**−0.62**	$<10^{-3}$
cal_high_cross_3	**−0.01**	$<10^{-3}$	**−0.01**	$<10^{-3}$	**−0.01**	$<10^{-3}$	**−0.00**	0.007	**−0.01**	$<10^{-3}$
cal_asymmetric_3	+0.00	–	+0.00	–	+0.00	–	+0.00	–	+0.00	–
duffing_triple_well	+0.00	–	+0.00	–	+0.00	–	+0.00	1.000	+0.00	–
arrested_spiral	+0.00	–	+0.00	–	+0.00	–	+0.00	–	+0.00	–
var_l_shape_5	+0.00	–	+0.00	–	+0.00	–	+0.00	–	+0.00	–
K/15 *Holm* $p < 0.05$	K=11 / N=11		K=11 / N=11		K=11 / N=11		K=11 / N=12		K=11 / N=11	

Table 14: Per-system effect sizes and Holm-corrected Wilcoxon p-values for the wrong-support-over-base MSE ratio at $H1$ on the deep slice. Alternative: candidate ratio > baseline.

System	LISTA		LISTA-BD		LISTA-SB		Sparse MLP, BD		Sparse MLP	
	Δ	p_{H}	Δ	p_{H}	Δ	p_{H}	Δ	p_{H}	Δ	p_{H}
cal_high_cross_3	$\mathbf{+4.98\times10^{3}}$	$<10^{-3}$	$\mathbf{+1.47\times10^{3}}$	$<10^{-3}$	$\mathbf{+6.29\times10^{3}}$	$<10^{-3}$	$\mathbf{+2.93\times10^{3}}$	$<10^{-3}$	$\mathbf{+3.16\times10^{3}}$	$<10^{-3}$
cal_pentagon_5	$\mathbf{+5.58\times10^{3}}$	$<10^{-3}$	$\mathbf{+5.22\times10^{3}}$	$<10^{-3}$	$\mathbf{+4.33\times10^{3}}$	$<10^{-3}$	$\mathbf{+1.36\times10^{3}}$	$<10^{-3}$	$\mathbf{+1.23\times10^{3}}$	$<10^{-3}$
var_depth_gradient_4	$\mathbf{+4.68\times10^{3}}$	$<10^{-3}$	$\mathbf{+4.57\times10^{3}}$	$<10^{-3}$	$\mathbf{+8.59\times10^{3}}$	$<10^{-3}$	$\mathbf{+1.04\times10^{3}}$	$<10^{-3}$	$\mathbf{+1.04\times10^{3}}$	$<10^{-3}$
cal_octagon_8	$\mathbf{+5.02\times10^{3}}$	$<10^{-3}$	$\mathbf{+6.47\times10^{3}}$	$<10^{-3}$	$\mathbf{+8.70\times10^{3}}$	$<10^{-3}$	$\mathbf{+3.83\times10^{3}}$	$<10^{-3}$	$\mathbf{+3.78\times10^{3}}$	$<10^{-3}$
gated_transfer_linear	$\mathbf{+3.67\times10^{3}}$	$<10^{-3}$	$\mathbf{+4.36\times10^{3}}$	$<10^{-3}$	$\mathbf{+5.44\times10^{3}}$	$<10^{-3}$	$\mathbf{+9.16\times10^{3}}$	$<10^{-3}$	$\mathbf{+7.65\times10^{3}}$	$<10^{-3}$
cal_square_4	$\mathbf{+1.06\times10^{4}}$	$<10^{-3}$	$\mathbf{+1.31\times10^{4}}$	$<10^{-3}$	$\mathbf{+1.71\times10^{4}}$	$<10^{-3}$	$\mathbf{+5.82\times10^{3}}$	$<10^{-3}$	$\mathbf{+5.40\times10^{3}}$	$<10^{-3}$
var_diamond_4	$\mathbf{+1.47\times10^{4}}$	$<10^{-3}$	$\mathbf{+5.46\times10^{3}}$	$<10^{-3}$	$\mathbf{+1.62\times10^{4}}$	$<10^{-3}$	$\mathbf{+8.87\times10^{3}}$	$<10^{-3}$	$\mathbf{+1.07\times10^{4}}$	$<10^{-3}$
cal_hexagon_6	$\mathbf{+1.33\times10^{4}}$	$<10^{-3}$	$\mathbf{+7.89\times10^{3}}$	$<10^{-3}$	$\mathbf{+9.78\times10^{3}}$	$<10^{-3}$	$\mathbf{+1.22\times10^{4}}$	$<10^{-3}$	$\mathbf{+1.22\times10^{4}}$	$<10^{-3}$
snic_multi	$\mathbf{+2.79\times10^{4}}$	$<10^{-3}$	$\mathbf{+1.85\times10^{4}}$	$<10^{-3}$	$\mathbf{+2.35\times10^{4}}$	$<10^{-3}$	$\mathbf{+2.09\times10^{5}}$	$<10^{-3}$	$\mathbf{+3.23\times10^{5}}$	$<10^{-3}$
transition_routes_4	$\mathbf{+7.07\times10^{4}}$	$<10^{-3}$	$\mathbf{+1.10\times10^{5}}$	$<10^{-3}$	$\mathbf{+7.64\times10^{4}}$	$<10^{-3}$	$\mathbf{+3.21\times10^{4}}$	$<10^{-3}$	$\mathbf{+2.75\times10^{4}}$	$<10^{-3}$
gated_local_linear	$\mathbf{+7.77\times10^{4}}$	$<10^{-3}$	$\mathbf{+5.51\times10^{4}}$	$<10^{-3}$	$\mathbf{+1.13\times10^{5}}$	$<10^{-3}$	$\mathbf{+2.81\times10^{5}}$	$<10^{-3}$	$\mathbf{+3.63\times10^{5}}$	$<10^{-3}$
K/15 *Holm* $p < 0.05$	K=11 / N=11		K=11 / N=11		K=11 / N=11		K=11 / N=11		K=11 / N=11	

Table 15: Per-system effect sizes and Holm-corrected Wilcoxon p-values for the wrong-support-over-base MSE ratio at $H20$. Alternative: candidate > baseline.

System	LISTA		LISTA-BD		LISTA-SB		Sparse MLP, BD		Sparse MLP	
	Δ	p_H	Δ	p_H	Δ	p_H	Δ	p_H	Δ	p_H
cal_high_cross_3	+1.91	0.076	−0.338	0.756	+0.790	0.094	**+3.46**	0.036	**+13.9**	0.008
var_depth_gradient_4	**+4.38**	0.035	**+0.919**	0.018	**+3.01**	$<10^{-3}$	**+13.6**	$<10^{-3}$	**+13.6**	$<10^{-3}$
cal_octagon_8	**+9.78**	$<10^{-3}$	**+4.45**	$<10^{-3}$	**+2.99**	$<10^{-3}$	**+6.57**	$<10^{-3}$	**+8.59**	$<10^{-3}$
cal_square_4	**+7.85**	$<10^{-3}$	**+9.39**	$<10^{-3}$	**+3.43**	$<10^{-3}$	**+10.6**	$<10^{-3}$	**+10.6**	$<10^{-3}$
cal_pentagon_5	**+11.0**	$<10^{-3}$	**+12.3**	$<10^{-3}$	**+7.70**	0.001	**+11.0**	$<10^{-3}$	**+8.97**	$<10^{-3}$
gated_transfer_linear	**+8.94**	0.002	**+22.3**	0.001	**+14.5**	$<10^{-3}$	**+27.5**	$<10^{-3}$	**+22.1**	$<10^{-3}$
cal_hexagon_6	**+29.3**	$<10^{-3}$	**+26.0**	$<10^{-3}$	**+7.23**	$<10^{-3}$	**+45.8**	$<10^{-3}$	**+45.8**	$<10^{-3}$
var_diamond_4	**+55.7**	$<10^{-3}$	**+1.75**	0.004	**+40.9**	$<10^{-3}$	**+49.4**	$<10^{-3}$	**+49.2**	$<10^{-3}$
transition_routes_4	**+308**	$<10^{-3}$	**+176**	$<10^{-3}$	**+214**	$<10^{-3}$	**+209**	$<10^{-3}$	**+154**	$<10^{-3}$
snic_multi	**+214**	$<10^{-3}$	**+282**	$<10^{-3}$	**+186**	$<10^{-3}$	**+573**	$<10^{-3}$	**+626**	$<10^{-3}$
gated_local_linear	**+568**	$<10^{-3}$	**+362**	$<10^{-3}$	**+1.17×10³**	$<10^{-3}$	**+2.95×10³**	$<10^{-3}$	**+4.07×10³**	$<10^{-3}$
$K/15$ *Holm* $p<0.05$	K=10 / N=11		K=10 / N=11		K=10 / N=11		K=11 / N=11		K=11 / N=11	

Table 16: Deep-state support-routed forecasting route-form diagnostics. S_{top8} columns use exact eight-index supports; F_{top8} columns use merged families of those exact supports. Entries are arithmetic means across systems after within-system seed-IQM summarization, with diagnostic within-system Wilcoxon/Holm counts in brackets. LISTA-SB is the matched d_z=256 soft-block LISTA ablation; lower is better.

*H*100

Model	Gated S_{top8} equation (34)	S_{top8}-local equation (14)	F_{top8}-local equation (14)
LISTA	0.783 [4/15]	0.771 [5/15]	0.165 [9/15]
LISTA-SB	0.861 [1/15]	0.861 [1/15]	0.00372 [13/15]
LISTA-BD	0.915 [1/15]	0.919 [1/15]	**0.00207** [12/15]
Sparse MLP, BD	**0.797** [0/15]	0.770 [0/15]	0.00941 [6/15]
Sparse MLP	0.813 [0/15]	**0.736** [0/15]	0.0104 [5/15]
Dense MLP	0.991 [0/15]	0.990 [0/15]	102 [0/15]

*H*1000

Model	Gated S_{top8} equation (34)	S_{top8}-local equation (14)	F_{top8}-local equation (14)
LISTA	0.681 [8/15]	0.676 [7/15]	$\mathbf{8.03\times10^{-14}}$ [11/15]
LISTA-SB	0.817 [7/15]	0.809 [6/15]	6.30×10^{-11} [13/15]
LISTA-BD	0.690 [6/15]	0.709 [7/15]	1.19×10^{-7} [13/15]
Sparse MLP, BD	$\mathbf{1.48\times10^{-5}}$ [1/15]	$\mathbf{2.99\times10^{-4}}$ [1/15]	1.49×10^{-6} [14/15]
Sparse MLP	0.804 [2/15]	0.738 [1/15]	0.126 [14/15]
Dense MLP	0.849 [0/15]	0.965 [0/15]	78.6 [0/15]

Table 17: Scale-normalized Dysts forecasting summary. Entries are arithmetic means across systems of the candidate per-system seed-IQM MSE divided by the Dense MLP per-system seed-IQM MSE; lower than 1 favors the candidate. This table gives the average-case ratio view corresponding to the actual-MSE table in table 3. LISTA-SB is retained as a diagnostic row.

Model	H100	H500	H1000	H1500	H2000	H3000	H4000	H5000
LISTA	0.509	0.464	0.459	0.426	0.440	0.489	0.565	0.595
LISTA-BD	0.530	0.587	0.502	0.445	0.443	0.498	0.574	0.612
LISTA-SB	0.972	1.026	1.019	1.088	1.157	1.264	1.086	1.006
Sparse MLP	0.462	0.499	0.501	0.563	0.566	0.610	0.685	0.739
Sparse MLP-BD	0.477	0.499	0.497	0.555	0.559	0.617	0.703	0.757
Dense MLP	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000